

DIABETIC RETINOPATHY DETECTION USING DEEP LEARNING

Project report submitted

for the partial fulfilment of the requirement for the award of the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

SURAPARAJU DURGA ANUSHA (17FE1A05F3)

SUNKARI SOWJANYA (17FE1A05C6)

SHAIK RESHMA (17FE1A05B9)

VELAGA BHARAT KUMAR (17FE1A05E0)

Under the Esteemed Guidance of

SK. GOUSE JOHN M.Tech,

Assistant Professor

Department of Computer Science and Engineering



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VIGNAN'S LARA INSTITUTE OF TECHNOLOGY AND SCIENCE

(Affiliated to Jawaharlal Nehru Technological University Kakinada, Kakinada)

(An ISO 9001:2015 Certified Institution , Approved by AICTE)

Vadlamudi, Guntur Dist, Andhra Pradesh- 522213

July-2021

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VIGNAN'S LARA INSTITUTE OF TECHNOLOGY AND SCIENCE

(Affiliated to Jawaharlal Nehru Technological University Kakinada, Kakinada)

(An ISO 9001:2015 Certified Institution , Approved by AICTE)

Vadlamudi-522213



CERTIFICATE

This is to certify that this project work entitled “**DIABETIC RETINOPATHY DETECTION USING DEEP LEARNING**” is the bonafide work done by **S.DURGA ANUSHA (17FE1A05F3), S.SOWJANYA (17FE1A05C6), SK.RESHMA (17FE1A05B9), V.BHARAT KUMAR (17FE1A05E0)** under my guidance and submitted in fulfilment of the requirements for the award of the degree of Bachelor of Technology in **COMPUTER SCIENCE AND ENGINEERING** from **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA, KAKINADA**. The work embodied in this project report is not submitted to any University or Institution for the award of any Degree or Diploma.

Project Guide

Mr. Sk. Gouse John M.Tech

Assistant Professor

Head of the Department

Dr. K. Venkateswara Rao Ph.D

Professor

External Examiner

DECLARATION

We hereby declare that the project report entitled “**DIABETIC RETINOPATHY DETECTION USING DEEP LEARNING**” is a record of an original work done by us under the guidance of **Mr. SK. GOUSE JOHN** , Assistant Professor of Computer Science and Engineering and this project report is submitted in the fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering. The results embodied in this project report are not submitted to any other University or Institute for the award of any Degree or Diploma.

	<u>Project Members</u>	<u>Signature</u>
Place: Vadlamudi	S.Durga Anusha(17FE1A05F3)	_____
Date:	S.Sowjanya(17FE1A05C6)	_____
	Sk.Reshma(17FE1A05B9)	_____
	V.Bharat Kumar(17FE1A05E0)	_____

ACKNOWLEDGEMENT

The satisfaction that accompanies with the successful completion of any task would be incomplete without the mention of people whose ceaseless cooperation made it possible, whose constant guidance and encouragement crown all efforts with success. We are grateful to **Mr. SK. GOUSE JOHN**, Assistant professor, Department of Computer Science and Engineering for guiding through this project and for encouraging right from the beginning of the project till successful completion of the project. Every interaction with him was an inspiration.

We thank **Dr. K. VENKATESWARA RAO**, Professor & HOD, Department of Computer Science and Engineering for support and Valuable suggestions.

We also express our thanks to **Dr. K. PHANEENDRA KUMAR**, Principal, Vignan's Lara Institute of Technology & Science for providing the resources to carry out the project.

We also express our sincere thanks to our beloved Chairman **Dr. LAVU RATHAIAH** for providing support and stimulating environment for developing the project.

We also place our floral gratitude to all other teaching and lab technicians for their constant support and advice throughout the project.

Project Members

S.Durga Anusha(17FE1A05F3)

S.Sowjanya(17FE1A05C6)

Sk.Reshma(17FE1A05B9)

V.Bharat Kumar(17FE1A05E0)

TABLE OF CONTENTS

LIST OF FIGURES	i-ii
LIST OF TABLES	iii
ABSTRACT	1
CHAPTER 1: INTRODUCTION	2-11
1.1 Diabetes	2
1.2 Anatomy of the Eye	3
1.3 Diabetic Retinopathy	4
1.4 Classification of DR	5-6
1.5 Diabetic Retinopathy Stages	7
1.6 Symptoms	8
1.7 Causes	9
1.8 Risk Factors	10-11
CHAPTER 2: LITERATURE SURVEY	12-13
2.1 Literature Study	12
2.2 Existing Methods	13
2.3 Limitations of Existing Methods	13
CHAPTER 3: PROPOSED METHODOLOGY	14-31
3.1 Introduction to Proposed Methodology	14
3.2 Proposed System	14
3.3 Convolutional Neural Network	14-16
3.3.1 Introduction to NN	14
3.3.2 Convolutional Neural Network	15
3.3.3 Types of Layers in ConvNet	16
3.4 Image Pre processing	17-23
3.4.1 Introduction	17
3.4.2 Pixel Brightness Transformation	18

3.4.3 Geometric Transformation	18
3.4.4 Image Filtering and Segmentation	21
3.4.5 Fourier Transform	23
3.5 Deep Learning Models	23-26
3.5.1 Introduction to Pre- Trained Models	23
3.5.2 ResNet152	24
3.5.3 DenseNet121	25
3.5.4 DenseNet169	26
3.6 Ensemble Approach	27-30
3.6.1 Introduction to Ensemble Approach	27
3.6.2 Stacking Approach	29
3.6.3 Weighted Average Ensemble	30
3.6.4 Model Averaging Ensemble	30
CHAPTER 4: EVALUATION METRICS	32-33
4.1 Definition	32
4.2 Confusion Matrix	32
4.3 Accuracy	32
4.4 Precision	33
4.5 Recall	33
4.6 F1-Score	33
4.7 AUC	33
CHAPTER 5: SYSTEM DESIGN	34-40
5.1 Block Diagram	34
5.2 UML Diagrams	35-40
5.2.1 Class Diagram	35
5.2.2 Use case Diagram	36
5.2.3 Sequence Diagram	37
5.2.4 State chart Diagram	38

5.2.5 Activity Diagram	39
5.2.6 Object Diagram	40
CHAPTER 6: TESTING	41-42
6.1 Testing Levels	41
6.2 System Test cases	42
CHAPTER 7: IMPLEMENTATION	43-59
CHAPTER 8: RESULTS	60-61
CHAPTER 9: CONCLUSION AND FUTURE SCOPE	62-63
9.1 Conclusion	62
9.2 Future Scope	63
REFERENCES	64-66

LIST OF FIGURES

Figure No	Figure Name	Page No
1.1	Anatomy of the eye	3
1.2	Types of Diabetic Retinopathy	5
1.3	NPDR	6
1.4	PDR	7
1.5	Comparison of normal eye and DR	9
3.1	Architecture of Neural Network	15
3.2	Convolution Operation	16
3.3	Max pooling	17
3.4	Layers in CNN	17
3.5	Spatial transformation	20
3.6	Fine tuning pre-trained models	24
3.7	Skip connections	25
3.8	ResNet152 architecture	25
3.9	DenseNet121 architecture	26
3.10	DenseNet169 architecture	27
3.11	Stacking	30
3.12	Weighted Average Ensemble	30
3.13	Model Averaging Ensemble	31
4.1	Confusion matrix	32
5.1	Block diagram of the model	34
5.2	Class Diagram	35
5.3	Use case Diagram	36
5.4	Sequence Diagram	37
5.5	State Chart Diagram	38

5.6	Activity Diagram	39
5.7	Object Diagram	40

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
6.1	Test case for images present in dataset	42
6.2	Test case for images not present in dataset	42

ABSTRACT

Diabetic Retinopathy is a diabetes related eye disease caused by high blood sugar levels. The high blood sugar levels cause the blood vessels in the retinas to swell, which leads to enlarged blood vessels and even vessel leaks, which lead to dark spots in retinal images. With this code, we aim to use the appearance of blood vessel leak spots as an indicator of background diabetic retinopathy, though further diagnosis techniques would be required in the real world. The goal of this code is to automate image processing and diagnosing of retinal images to identify stages of diabetic retinopathy using Deep Learning.

1805 normal retinal images and 1857 diagnosed retinal images were processed through a code that first reads and filters the images and then determine if diabetic retinopathy is present. The results are then printed onto the command window for viewer interpretation. The accuracy obtained is 0.875 for ensemble approach of DenseNet121, DenseNet169, ResNet152.

CHAPTER-1

INTRODUCTION

In this chapter, the diabetic impediment in the eye and its significance to the vision is presented. The anatomical background about the functioning of the eye and various modalities available to assess the eye is presented in this Chapter. The discussion about the diabetic retinopathy, different retina anomalies and the diagnostic methods is also presented.

1.1 Introduction to Diabetes

Diabetes is a disease caused by increase of the glucose level in the blood. This disease is becoming the major health hazard across the world. It has various adverse consequences on almost all organs of the patient. The retina of the eye is the most rapidly and likely affected organ by the diabetes. It is one of the major sources of becoming the patients blind and visually impaired. As per the survey conducted by World Diabetes Foundation, by 2030 over 439 million people of the world will be affected by diabetes. The use of digital image processing technology in the medical field has become very popular for multipurpose applications. The digital imaging machinery nowadays is used in almost all the diagnosis and treatment related areas. The modern eye care experts are using fundus cameras to capture images of the eye. Imaging the retinal fundus of the eye is a non-invasive process. The proper investigation of the eye disease due to diabetes should be conducted by expert ophthalmologists. Regular screening of the eye of the patients is essential for the prevention of the eye disease due to diabetes. Diabetic retinopathy (DR) is vision threatening disease caused by diabetes. The existing screening facilities and expert manpower is not sufficient for the current scenario. The development of an automatic eye fundus investigation techniques are extremely needed to deal with the higher demand of the screening capabilities. There is a huge requirement of automatic detection of DR. The automatic eye fundus investigation techniques will save lot of time and workload of the expert ophthalmologists.

The main objective of this thesis is to develop an automatic eye fundus investigation technique using digital fundus images which will be more accurate and computationally efficient for screening of DR. This chapter describes the effect of diabetes on the eye, the structure and functioning of the eye and different retina abnormalities. The lesions that are directly caused by diabetes to retina are presented here. The screening process of DR is also discussed. The present and potential scenarios for the early detection of DR are also discussed in this chapter.

1.2 Anatomy of the eye:

Vision occurs when light enters the eye through the pupil. With help from other important structures in the eye, like the iris and cornea, the appropriate amount of light is directed towards the lens. Just like a lens in a camera sends a message to produce a film, the lens in the eye 'refracts' (bends) incoming light onto the retina. The retina is made up by millions of specialised cells known as rods and cones, which work together to transform the image into electrical energy, which is sent to the optic disk on the retina and transferred via electrical impulses along the optic nerve to be processed by the brain.

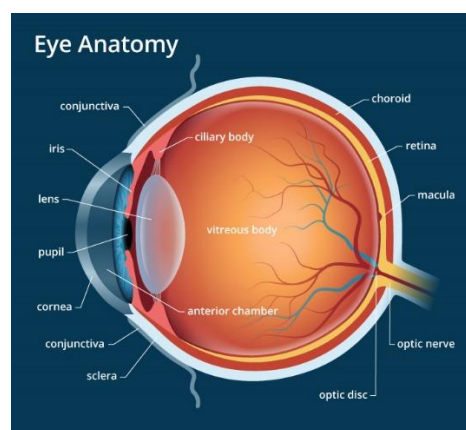


Fig 1.1 Anatomy of the eye

- **Iris:** regulates the amount of light that enters your eye. It forms the coloured, visible part of your eye in front of the lens. Light enters through a central opening called the pupil.
- **Pupil:** the circular opening in the centre of the iris through which light passes into the lens of the eye. The iris controls widening and narrowing (dilation and constriction) of the pupil.
- **Cornea:** the transparent circular part of the front of the eyeball. It refracts the light entering the eye onto the lens, which then focuses it onto the retina. The cornea contains no blood vessels and is extremely sensitive to pain.
- **Lens:** a transparent structure situated behind your pupil. It is enclosed in a thin transparent capsule and helps to refract incoming light and focus it onto the retina. A cataract is when the lens becomes cloudy, and a cataract operation involves the replacement of the cloudy lens with an artificial plastic lens.
- **Choroid:** the middle layer of the eye between the retina and the sclera. It also contains a pigment that absorbs excess light so preventing blurring of vision.
- **Ciliary body:** the part of the eye that connects the choroid to the iris.
- **Retina:** a light sensitive layer that lines the interior of the eye. It is composed of light sensitive cells known as rods and cones. The human eye contains about 125 million rods, which are necessary for seeing in dim light. Cones, on the other hand,

function best in bright light. There are between 6 and 7 million cones in the eye and they are essential for receiving a sharp accurate image and for distinguishing colours. The retina works much in the same way as film in a camera.

- **Macula:** a yellow spot on the retina at the back of the eye which surrounds the fovea.
- **Fovea:** forms a small indentation at the centre of the macula and is the area with the greatest concentration of cone cells. When the eye is directed at an object, the part of the image that is focused on the fovea is the image most accurately registered by the brain.
- **Optic disc:** the visible (when the eye is examined) portion of the optic nerve, also found on the retina. The optic disc identifies the start of the optic nerve where messages from cone and rod cells leave the eye via nerve fibres to the optic centre of the brain. This area is also known as the 'blind spot'.
- **Optic nerve:** leaves the eye at the optic disc and transfers all the visual information to the brain.
- **Sclera:** the white part of the eye, a tough covering with which the cornea forms the external protective coat of the eye.
- **Rod cells** are one of the two types of light-sensitive cells in the retina of the eye. There are about 125 million rods, which are necessary for seeing in dim light.
- **Cone cells** are the second type of light sensitive cells in the retina of the eye. The human retina contains between six and seven million cones; they function best in bright light and are essential for acute vision (receiving a sharp accurate image). It is thought that there are three types of cones, each sensitive to the wavelength of a different primary colour – red, green or blue. Other colours are seen as combinations of these primary colours.

1.3 DIABETIC RETINOPATHY:

Diabetic retinopathy is an eye condition that can cause vision loss and blindness in people who have diabetes. It affects blood vessels in the retina (the light-sensitive layer of tissue in the back of your eye).

If you have diabetes, it's important for you to get a comprehensive dilated eye exam at least once a year. Diabetic retinopathy may not have any symptoms at first — but finding it early can help you take steps to protect your vision.

Managing your diabetes by staying physically active, eating healthy, and taking your medicine can also help you prevent or delay vision loss.

1.4 Classification of Diabetic Retinopathy:

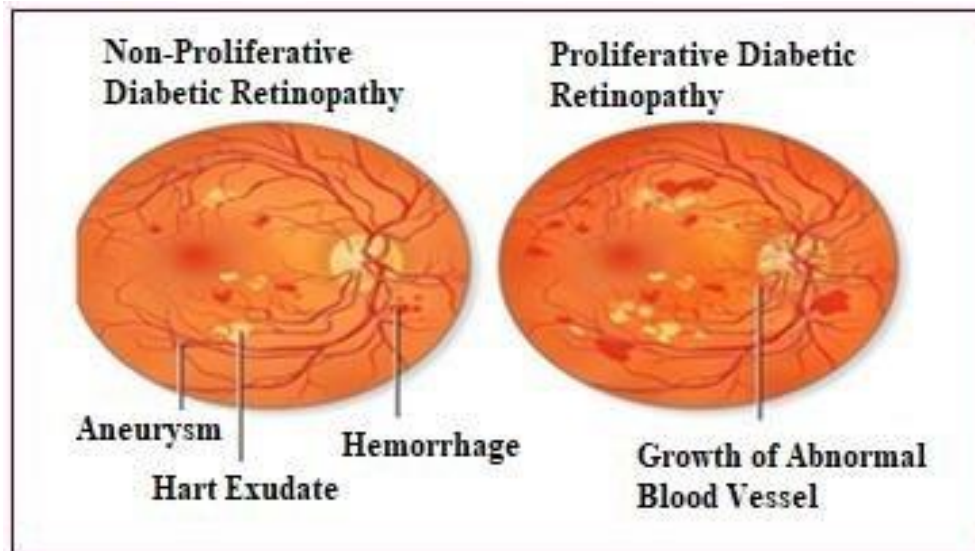


Fig 1.2 Types of Diabetic Retinopathy

Diabetic retinopathy falls into two main classes: non-proliferative and proliferative. The word "proliferative" refers to whether or not there is neovascularization (abnormal blood vessel growth) in the retina. Early disease without neovascularization is called non-proliferative diabetic retinopathy (NPDR). As the disease progresses, it may evolve into proliferative diabetic retinopathy (PDR), which is defined by the presence of neovascularization and has a greater potential for serious visual consequences.

NPDR – Hyper-glycemia results in damage to retinal capillaries. This weakens the capillary walls and results in small outpouchings of the vessel lumens, known as microaneurysms. Microaneurysms eventually rupture to form haemorrhages deep within the retina, confined by the internal limiting membrane (ILM). Because of their dot-like appearance, they are called "dot-and-blot" haemorrhages.

The weakened vessels also become leaky, causing fluid to seep into the retina. Fluid deposition under the macula, or macular edema, interferes with the macula's normal function and is a common cause of vision loss in those with DR. Resolution of fluid lakes can leave behind sediment, similar to a receding river after a flood. This sediment is composed of lipid by products and appears as waxy, yellow deposits called hard exudates. As NPDR progresses, the affected vessels eventually become obstructed. This obstruction may cause infarction of the nerve fiber layer, resulting in fluffy, white patches called cotton wool spots (CWS).



Fig 1.3 NPDR

NPDR is further subdivided based on retinal findings:

Early NPDR – At least one microaneurysm present on retinal exam.

Moderate NPDR – Characterized by multiple microaneurysms, dot-and-blot hemorrhages, venous beading, and/or cotton wool spots.

Severe NPDR – In the most severe stage of NPDR, you will find cotton wool spots, venous beading, and severe intraretinal microvascular abnormalities (IRMA). It is diagnosed using the "4-2-1 rule." A diagnosis is made if the patient has any of the following: diffuse intraretinal hemorrhages and microaneurysms in 4 quadrants, venous beading in ≥ 2 quadrants, or IRMA in ≥ 1 quadrant. Within one year, 52-75% of patients falling into this category will progress to PDR.

PDR – As mentioned earlier, the retina has a high metabolic requirement, so with continued ischemia, retinal cells respond by releasing angiogenic signals such as vascular endothelial growth factor (VEGF). Angiogenic factors, like VEGF, stimulate growth of new retinal blood vessels to bypass the damaged vessels. This is referred to as neovascularization. In PDR, the fibrovascular proliferation extends beyond the ILM. This may sound like a good idea, but the new vessels are leaky, fragile, and often misdirected. They may even grow off the retina and into the vitreous. As the vitreous shrinks with age, it pulls on these fragile vessels and can cause them to tear, resulting in a vitreous hemorrhage and sudden vision loss. These vessels may also scar down, forming strong anchors between the retina and vitreous causing traction on the retina. If enough force is created, a tractional retinal detachment may occur. This is another mechanism by which DR can cause sudden vision loss. If the retina is not re-attached soon, especially if the macula is involved, vision may be permanently compromised.

While the effects of neovascularization in PDR can be devastating, the most common cause of vision loss in diabetics is macular edema. Macular edema can occur in NPDR, but it is more common in more severe cases of DR due to the leakiness of the new blood vessels.

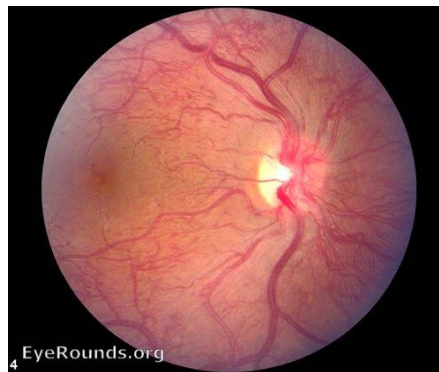


Fig 1.4 PDR

Diabetics can also have problems located more anteriorly in the eye. The angiogenic molecules that are produced by the retina may float anteriorly, causing neovascularization of the iris. These vessels can grow into the angle of the anterior chamber where the trabecular meshwork, the drain of the eye, resides. This can obstruct outflow of aqueous fluid, raising intraocular pressure and causing acute glaucoma.

1.5 Diabetic Retinopathy stages:

In Diabetic Retinopathy there are having Four stages. They are

- Mild Non-proliferative Retinopathy (MNPR)
- Moderate Non-proliferative Retinopathy (MNPR)
- Severe Non-proliferative Retinopathy (SNPR)
- Proliferative Diabetic Retinopathy (PDR)

Mild Non-proliferative Retinopathy (MNPR):

The first stage is also called background retinopathy. It means that there are tiny bulges in the tiny blood vessels in your retinas. The bulges are called microaneurysms. They may cause the vessels to leak small amounts of blood into your retinas.

At this early stage, you probably don't have vision problems, so you may not need treatment. Talk to your doctor about ways to keep your condition from getting worse. You'll need to keep your blood sugar, blood pressure, and cholesterol levels under control. You should plan to have another screening test in 12 months.

If the doctor diagnoses you with this condition in both eyes, you have a 25% chance of progressing to the third stage in the next 3 years.

Moderate Non-proliferative Retinopathy:

The second stage is also called pre-proliferative retinopathy. At this stage, the blood vessels in your retinas swell. They may not carry blood as well as they used to. These things can cause physical changes to the retina.

These changes can lead to diabetic macular edema (DME). This happens when blood and other fluids build up in a part of your retina called the macula. The macula is key for straight-ahead vision, like when you read or drive. When it swells, it can cause problems with this important part of your vision.

Half the people with diabetic retinopathy will get DME. It can happen at any stage, but it's more likely to happen as the condition advances. Reaching this stage means a greater chance that the disease will affect your vision. Your doctor may recommend eye tests every 3 to 6 months.

Severe Non-proliferative Retinopathy:

This is also called proliferative retinopathy. In this stage, your blood vessels become even more blocked. This means even less blood goes to the retinas. Because of this, scar tissue forms. The lack of blood triggers a signal to your retinas to create new blood vessels.

If the blood vessels close off completely, it's called macular ischemia. This can lead to blurry vision with dark spots some people describe as "floaters".

If you reach this stage, there's a very high chance that you'll lose your vision. Treatment may be able to stop further vision loss. But if you've already lost some of your vision, it's unlikely to come back.

Proliferative Diabetic Retinopathy:

In this advanced stage, new blood vessels grow in your retinas and into the gel-like fluid that fills your eyes. This growth is called neovascularization. These vessels are thin and weak. They often bleed. The bleeding can cause scar tissue.

As the scar tissue gets smaller, it can pull the retina away from the back of your eye. This is called retinal detachment. It can lead to permanent loss of both straight-ahead and side vision.

1.6 Symptoms

You might not have symptoms in the early stages of diabetic retinopathy. As the condition progresses, diabetic retinopathy symptoms may include:

- Spots or dark strings floating in your vision (floaters)
- Blurred vision

- Fluctuating vision
- Impaired color vision
- Dark or empty areas in your vision
- Vision loss

1.7 Causes

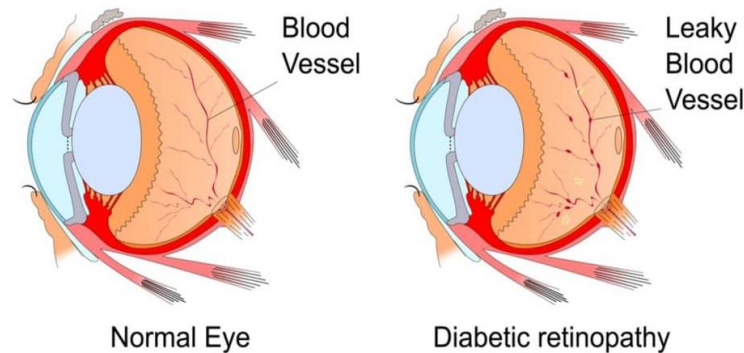


Fig 1.5 Comparison of Normal Eye and Diabetic Retinopathy

Over time, too much sugar in your blood can lead to the blockage of the tiny blood vessels that nourish the retina, cutting off its blood supply. As a result, the eye attempts to grow new blood vessels. But these new blood vessels don't develop properly and can leak easily.

There are two types of diabetic retinopathy:

- **Early diabetic retinopathy-** In this more common form — called non-proliferative diabetic retinopathy (NPDR) — new blood vessels aren't growing (proliferating). When you have NPDR, the walls of the blood vessels in your retina weaken. Tiny bulges (microaneurysms) protrude from the vessel walls of the smaller vessels, sometimes leaking fluid and blood into the retina. Larger retinal vessels can begin to dilate and become irregular in diameter, as well. NPDR can progress from mild to severe, as more blood vessels become blocked. Nerve fibers in the retina may begin to swell. Sometimes the central part of the retina (macula) begins to swell (macular edema), a condition that requires treatment.
- **Advanced diabetic retinopathy-** Diabetic retinopathy can progress to this more severe type, known as proliferative diabetic retinopathy. In this type, damaged blood vessels close off, causing the growth of new, abnormal blood vessels in the retina, and can leak into the clear, jelly-like substance that fills the center of your eye (vitreous). Eventually, scar tissue stimulated by the growth of new blood vessels may cause the retina to detach from the back of your eye. If the new blood vessels interfere with

the normal flow of fluid out of the eye, pressure may build up in the eyeball. This can damage the nerve that carries images from your eye to your brain (optic nerve), resulting in glaucoma.

1.8 Risk factors

Anyone who has diabetes can develop diabetic retinopathy. Risk of developing the eye condition can increase as a result of:

- Duration of diabetes — the longer you have diabetes, the greater your risk of developing diabetic retinopathy
- Poor control of your blood sugar level
- High blood pressure
- High cholesterol
- Pregnancy
- Tobacco use
- Being African-American, Hispanic or Native American

Complications

Diabetic retinopathy involves the abnormal growth of blood vessels in the retina. Complications can lead to serious vision problems:

- **Vitreous hemorrhage-** The new blood vessels may bleed into the clear, jelly-like substance that fills the center of your eye. If the amount of bleeding is small, you might see only a few dark spots (floaters). In more-severe cases, blood can fill the vitreous cavity and completely block your vision.
Vitreous hemorrhage by itself usually doesn't cause permanent vision loss. The blood often clears from the eye within a few weeks or months. Unless your retina is damaged, your vision may return to its previous clarity.
- **Retinal detachment-** The abnormal blood vessels associated with diabetic retinopathy stimulate the growth of scar tissue, which can pull the retina away from the back of the eye. This may cause spots floating in your vision, flashes of light or severe vision loss.
- **Glaucoma-** New blood vessels may grow in the front part of your eye and interfere with the normal flow of fluid out of the eye, causing pressure in the eye to build up (glaucoma). This pressure can damage the nerve that carries images from your eye to your brain (optic nerve).
- **Blindness-** Eventually, diabetic retinopathy, glaucoma or both can lead to complete vision loss.

Prevention

You can't always prevent diabetic retinopathy. However, regular eye exams, good control of your blood sugar and blood pressure, and early intervention for vision problems can help prevent severe vision loss.

If you have diabetes, reduce your risk of getting diabetic retinopathy by doing the following:

- **Manage your diabetes.**
Make healthy eating and physical activity part of your daily routine. Try to get at least 150 minutes of moderate aerobic activity, such as walking, each week. Take oral diabetes medications or insulin as directed.
- **Monitor your blood sugar level.**
You may need to check and record your blood sugar level several times a day — more-frequent measurements may be required if you're ill or under stress. Ask your doctor how often you need to test your blood sugar.
- **Ask your doctor about a glycosylated haemoglobin test.**
The glycosylated haemoglobin test, or haemoglobin A1C test, reflects your average blood sugar level for the two- to three-month period before the test. For most people, the A1C goal is to be under 7 percent.
- **Keep your blood pressure and cholesterol under control.**
Eating healthy foods, exercising regularly and losing excess weight can help. Sometimes medication is needed, too.
- **If you smoke or use other types of tobacco, ask your doctor to help you quit.**
Smoking increases your risk of various diabetes complications, including diabetic retinopathy.
- **Pay attention to vision changes.**
Contact your eye doctor right away if you experience sudden vision changes or your vision becomes blurry, spotty or hazy.

Remember, diabetes doesn't necessarily lead to vision loss. Taking an active role in diabetes management can go a long way toward preventing complications.

CHAPTER-2

LITERATURE SURVEY

2.1 Literature Study

[1] They used the publicly available Kaggle dataset of retina images to train an ensemble of five deep Convolution Neural Network (CNN) models (Resnet50, Inceptionv3, Xception, Dense121, Dense169) to encode the rich features and improve the classification for different stages of DR. The dataset they have used is Kaggle dataset and they got an accuracy of 80%.

[2] They proposed a deep learning method for interpretable diabetic retinopathy (DR) detection. The visual-interpretable feature of the proposed method is achieved by adding the regression activation map (RAM) after the global averaging pooling layer of the convolutional networks (CNN). With RAM, the proposed model can localize the discriminative regions of an retina image to show the specific region of interest in terms of its severity level. The advantage of the proposed deep learning model is highly desired for DR detection because in practice, users are not only interested with high prediction performance, but also keen to understand the insights of DR detection and why the adopted learning model works.

[3] This study proposes a machine learning method for extracting three features like exudates, hemorrhages, and micro aneurysms and classification using hybrid classifier which is a combination of support vector machine, k nearest neighbour, random forest, logistic regression, multilayer perceptron network. From the results of the experiments, the highest accuracy values 82%. Hybrid approach produced a precision score of 0.8119, Recall score of 0.8116 and f-measure score of 0.8028.

[4] Throughout this paper, a deep Convolutional Neural Network (CNN) with 18 convolutional layers and 3 fully connected layers is proposed to analyze fundus images and automatically distinguish between controls (i.e. no DR), moderate DR (i.e. a combination of mild and moderate Non Proliferative DR (NPDR)) and severe DR (i.e. a group of severe NPDR, and Proliferative DR (PDR)) with a validation accuracy of 82%-83%, a sensitivity of 87%-89%, a specificity of 94%-95%, and a Quadratic Weighted Kappa Score of 0.91–0.92 when both 5-fold, and 10-fold cross validation methods were used respectively.

2.2 Existing Methods

The existing work classifies Diabetic Retinopathy in different ways. One of them is collecting the data about different features of retina and classifying the DR stage. From the technical point of view, We came through several machine learning models that train on the retinal data and can further be used to predict diabetic retinopathy.

Several existing methodologies for diabetic retinopathy detection are:

- The machine learning methods that take retinal data in the form of csv file and gets trained on it for classification.
- Fundus images of retina is best fit input for DR detection. CNN models are trained using fundus image dataset and these models are best to find only the early stages of the disease.

2.3 LIMITATIONS IN EXISTING METHODS

- Huge number of wrong predictions are found when fundus images of retina are manually examined.
- Due to unavailability of balanced datasets, Models fail to predict early stages of diabetic retinopathy efficiently.
- Images should be finely pre-processed in order to train and test the model efficiently.

CHAPTER- 3

PROPOSED METHODOLOGY

3.1 Introduction to Proposed Methodology

In general, diagnosis of Diabetic Retinopathy is done by examining the fundus images of Retina. But, it is error prone and difficult. Now a days, different technologies like machine learning and deep learning are used to diagnosis different diseases. In this, we tried to examine the fundus images using different deep learning techniques and select best fit models and ensemble them to make the model more efficient.

3.2 Proposed System

We have considered a dataset which contains fundus images of retina and we have also referred few research papers about the disease diagnosis and different deep learning and machine learning techniques that can be used for the detection of stages.

First, We have created image data generator that rescales, flips and splits the images and then dataset is splitted into training and validation sets.

Then we have tried to predict DR stage in different ways.

1. We considered all the images of dataset and applied deep learning techniques to predict 5 stages .
2. We considered entire dataset and merged the stages such that the resultant dataset consists of only 2 stages i.e., DR and No DR.
3. We have reduced the dataset size inorder to make it more balanced and then applied different deep learning models.
4. Ensembled the best fit models that are observed in the 3rd way.

3.3 Convolutional Neural Network

3.3.1 Introduction to Neural Network

Artificial Neural Networks are used in various classification task like image, audio, words. Different types of Neural Networks are used for different purposes, for example for predicting the sequence of words we use Recurrent Neural Networks more precisely an LSTM, similarly for image classification we use Convolution Neural Network.

In a regular Neural Network, there are three types of layers:

1. **Input Layers:** It's the layer in which we give input to our model. The number of neurons in this layer is equal to total number of features in our data (number of pixels incase of an image).
2. **Hidden Layer:** The input from Input layer is then feed into the hidden layer. There can be many hidden layers depending upon our model and data size. Each hidden layers can have different numbers of neurons which are generally greater than the number of features. The output from each layer is computed by matrix multiplication of output of the previous layer with learnable weights of that layer and then by addition of learnable biases followed by activation function which makes the network nonlinear.
3. **Output Layer:** The output from the hidden layer is then fed into a logistic function like sigmoid or softmax which converts the output of each class into probability score of each class.

The data is then fed into the model and output from each layer is obtained this step is called feedforward, we then calculate the error using an error function, some common error functions are cross entropy, square loss error etc. After that, we backpropagate into the model by calculating the derivatives. This step is called Backpropagation which basically is used to minimize the loss.

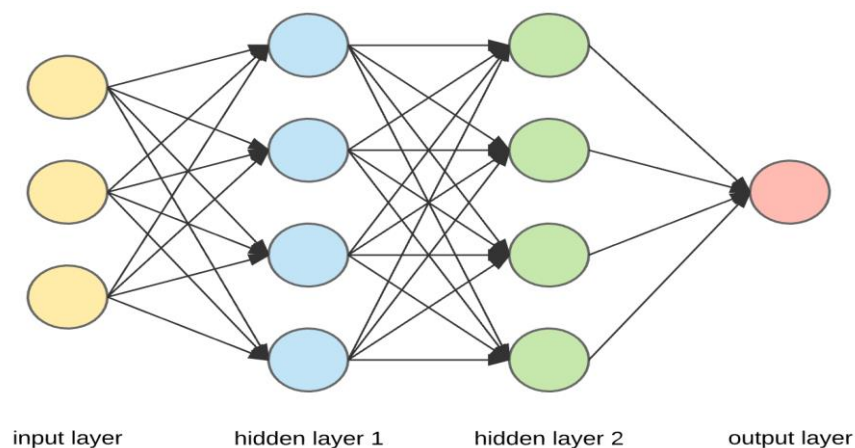


Fig 3.1 Architecture of Neural Network

3.3.2 Convolutional Neural Network

Convolution Neural Networks or convnets are neural networks that share their parameters. Imagine you have an image. It can be represented as a cuboid having its length, width (dimension of the image) and height (as image generally have red, green, and blue channels).

- Convolution layers consist of a set of learnable filters (patch in the above image). Every filter has small width and height and the same depth as that of input volume (3 if the input layer is image input).

- For example, if we have to run convolution on an image with dimension $34 \times 34 \times 3$. Possible size of filters can be $a \times a \times 3$, where 'a' can be 3, 5, 7, etc but small as compared to image dimension.
- During forward pass, we slide each filter across the whole input volume step by step where each step is called stride (which can have value 2 or 3 or even 4 for high dimensional images) and compute the dot product between the weights of filters and patch from input volume.
- As we slide our filters we'll get a 2-D output for each filter and we'll stack them together and as a result, we'll get output volume having a depth equal to the number of filters. The network will learn all the filters.

3.3.3 Types of Layers in ConvNet

A convnets is a sequence of layers, and every layer transforms one volume to another through differentiable function.

Let's take an example by running a convnets on of image of dimension $32 \times 32 \times 3$.

1. **Input Layer:** This layer holds the raw input of image with width 32, height 32 and depth 3.
2. **Convolution Layer:** This layer computes the output volume by computing dot product between all filters and image patch. Suppose we use total 12 filters for this layer we'll get output volume of dimension $32 \times 32 \times 12$.

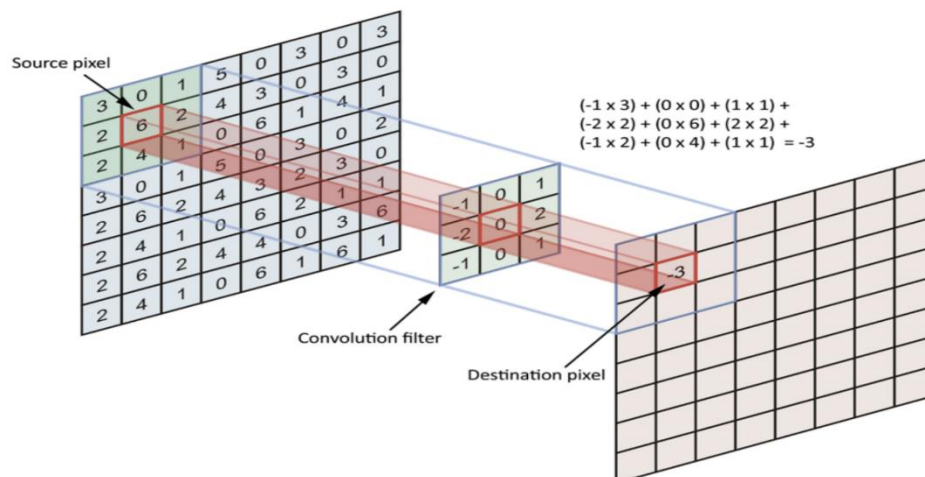


Fig 3.2 Convolution Operation

3. **Activation Function Layer:** This layer will apply element wise activation function to the output of convolution layer.

Some common activation functions are

- RELU: $\max(0, x)$
- Sigmoid: $1/(1+e^{-x})$
- Tanh, Leaky RELU, etc.

The volume remains unchanged hence output volume will have dimension $32 \times 32 \times 12$.

4. **Pool Layer:** This layer is periodically inserted in the convnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents from overfitting. Two common types of pooling layers are **max**

pooling and **average pooling**. If we use a max pool with 2 x 2 filters and stride 2, the resultant volume will be of dimension 16x16x12.

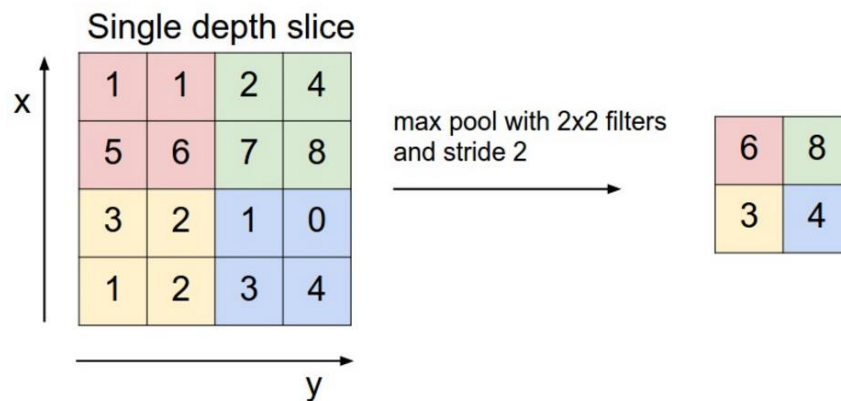


Fig 3.3 Max pooling

5. **Fully-Connected Layer:** This layer is regular neural network layer which takes input from the previous layer and computes the class scores and outputs the 1-D array of size equal to the number of classes.

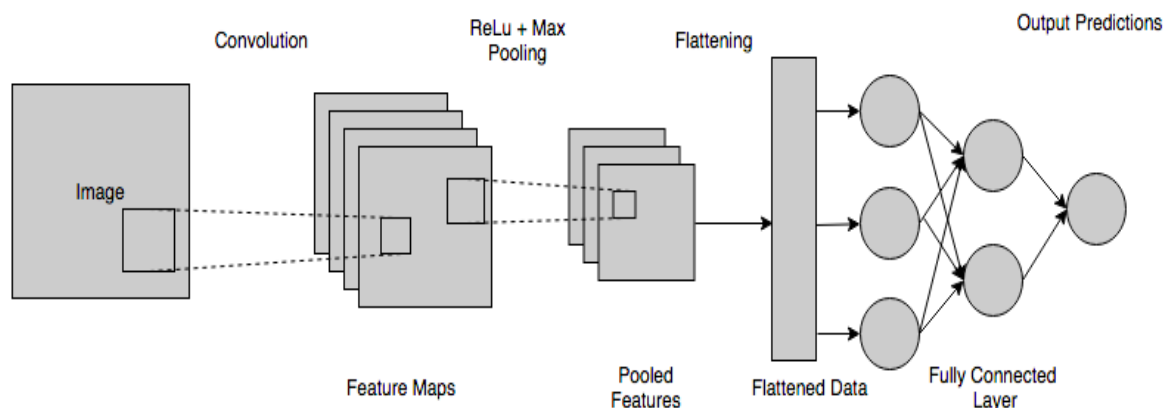


Fig 3.4 Layers in Convolutional Neural Network

3.4 IMAGE PREPROCESSING

3.4.1 Introduction

Image pre-processing is the term for operations on images at the lowest level of abstraction. These operations do not increase image information content but they decrease it if entropy is an information measure. The aim of pre-processing is an improvement of the image data that suppresses undesired distortions or enhances some image features relevant for further processing and analysis task.

There are 4 different types of Image Pre-Processing techniques and they are listed below.

1. Pixel brightness transformations/ Brightness corrections
2. Geometric Transformations
3. Image Filtering and Segmentation
4. Fourier transform and Image restoration

3.4.2 Pixel brightness transformations(PBT)

Brightness transformations modify pixel brightness and the transformation depends on the properties of a pixel itself. In PBT, output pixel's value depends only on the corresponding input pixel value. Examples of such operators include brightness and contrast adjustments as well as colour correction and transformations.

Contrast enhancement is an important area in image processing for both human and computer vision. It is widely used for medical image processing and as a pre-processing step in speech recognition, texture synthesis, and many other image/video processing applications

There are two types of Brightness transformations and they are below.

1. Brightness corrections
2. Gray scale transformation

3.4.3 Geometric Transformations

The earlier methods in this article deal with the colour and brightness/contrast. With geometric transformation, positions of pixels in an image are modified but the colours are unchanged.

Geometric transforms permit the elimination of geometric distortion that occurs when an image is captured. The normal Geometric transformation operations are rotation, scaling and distortion (or undistortion!) of images.

There are two basic steps in geometric transformations:

1. Spatial transformation of the physical rearrangement of pixels in the image
2. Grey level interpolation, which assigns grey levels to the transformed image

Transformations :

1. **Scaling** : Scaling is just resizing of the image

$$\begin{aligned} x' &= x \cdot S_x \\ y' &= y \cdot S_y \end{aligned} \Rightarrow \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

2. **Translation** : Translation is the shifting of object's location

$$\begin{aligned} x' &= x + \Delta x \\ y' &= y + \Delta y \end{aligned} \Rightarrow \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

3. **Rotation** : Just rotating an object with theta degrees

$$\begin{aligned} x' &= x \cdot \cos \theta - y \cdot \sin \theta \\ y' &= x \cdot \sin \theta + y \cdot \cos \theta \end{aligned} \Rightarrow \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

4. **Shearing** : Shifting the pixels horizontally

$$\begin{aligned} x' &= x + y \cdot B_x \\ y' &= x \cdot B_y + y \end{aligned} \Rightarrow \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & B_x \\ B_y & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

5. **Affine Transformation** : Instead of defining the scale factors, the shearing factors and the rotation angle, it is common to merge these three transformation into one matrix. The combination of the four transformations is therefore defined as Affine Transformation

$$\begin{aligned} x' &= a_1 \cdot x + a_2 \cdot y + a_3 \\ y' &= b_1 \cdot x + b_2 \cdot y + b_3 \end{aligned} \Rightarrow \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_1 & a_2 \\ b_1 & b_2 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} a_3 \\ b_3 \end{bmatrix}$$

6. **Perspective Transformation** : change the perspective of a given image or video for getting better insights about the required information. Here the points needs to be provided on the image from which want to gather information by changing perspective.



Fig 3.5 Spatial Transformations

Interpolation Methods :

After the transformation methods, the new point co-ordinates (x' , y') were obtained. Let's suppose these new points do not in general fit the discrete raster of the output image. So Each pixel value in the output image raster can be obtained by interpolation methods.

The brightness interpolation problem is usually expressed in a dual way. The brightness value of the pixel (x', y') in the output image where x' and y' lie on the discrete raster and it is

$$(x, y) = \mathbf{T}^{-1}(x', y')$$

Different types of Interpolation methods are

- Nearest neighbour interpolation is the simplest technique that re samples the pixel values present in the input vector or a matrix

$$f_1(x, y) = g_s(\text{round}(x), \text{round}(y))$$

- Linear interpolation explores four points neighbouring the point (x,y), and assumes that the brightness function is linear in this neighbourhood.

$$\begin{aligned} f_2(x, y) = & (1 - a)(1 - b) g_s(l, k) \\ & + a(1 - b) g_s(l + 1, k) \\ & + b(1 - a) g_s(l, k + 1) \\ & + ab g_s(l + 1, k + 1) \end{aligned}$$

$$\begin{aligned} l = \text{floor}(x), \quad a = x - l \\ k = \text{floor}(y), \quad b = y - k \end{aligned}$$

- Bicubic interpolation improves the model of the brightness function by approximating it locally by a bicubic polynomial surface. Sixteen neighbouring points are used for interpolation.

$$h_3 = \begin{cases} 1 - 2|x|^2 + |x|^3 & \text{for } 0 < |x| < 1 \\ 4 - 8|x| + 5|x|^2 - |x|^3 & \text{for } 1 < |x| < 2 \\ 0 & \text{otherwise} \end{cases}$$

3.4.4 Image Filtering and Segmentation

The goal of using filters is to modify or enhance image properties and/or to extract valuable information from the pictures such as edges, corners, and blobs. A filter is defined by a kernel, which is a small array applied to each pixel and its neighbors within an image

Some of the basic filtering techniques are

Low Pass Filtering (Smoothing) : A low pass filter is the basis for most smoothing methods. An image is smoothed by decreasing the disparity between pixel values by averaging nearby pixels

1. **High pass filters** (Edge Detection, Sharpening) : High-pass filter can be used to make an image appear sharper. These filters emphasize fine details in the image – the opposite of the low-pass filter. High-pass filtering works in the same way as low-pass filtering; it just uses a different convolution kernel.
2. **Directional Filtering** : Directional filter is an edge detector that can be used to compute the first derivatives of an image. The first derivatives (or slopes) are most evident when a large change occurs between adjacent pixel values. Directional filters can be designed for any direction within a given space
3. **Laplacian Filtering** : Laplacian filter is an edge detector used to compute the second derivatives of an image, measuring the rate at which the first derivatives change. This determines if a change in adjacent pixel values is from an edge or continuous progression. Laplacian filter kernels usually contain negative values in a cross pattern, centered within the array. The corners are either zero or positive values. The center value can be either negative or positive.

Image Segmentation

Image segmentation is a commonly used technique in digital image processing and analysis to partition an image into multiple parts or regions, often based on the characteristics of the pixels in the image. Image segmentation could involve separating foreground from background, or clustering regions of pixels based on similarities in colour or shape.

Image Segmentation mainly used in

- Face detection
- Medical imaging
- Machine vision
- Autonomous Driving

There are two types of image segmentation techniques.

Non-contextual thresholding: Thresholding is the simplest non-contextual segmentation technique. With a single threshold, it transforms a greyscale or colour image into a binary image considered as a binary region map. The binary map contains two possibly disjoint regions, one of them containing pixels with input data values smaller than a threshold and another relating to the input values that are at or above the threshold. The below are the types of thresholding techniques

1. Simple thresholding
2. Adaptive thresholding

3. Colour thresholding

3.4.5 Fourier transform

The Fourier Transform is an important image processing tool which is used to decompose an image into its sine and cosine components. The output of the transformation represents the image in the Fourier or frequency domain, while the input image is the spatial domain equivalent. In the Fourier domain image, each point represents a particular frequency contained in the spatial domain image.

The Fourier Transform is used in a wide range of applications, such as image analysis, image filtering, image reconstruction and image compression.

The DFT(Discrete Fourier Transform) is the sampled Fourier Transform and therefore does not contain all frequencies forming an image, but only a set of samples which is large enough to fully describe the spatial domain image. The number of frequencies corresponds to the number of pixels in the spatial domain image, i.e. the image in the spatial and Fourier domain are of the same size.

For a square image of size $N \times N$, the two-dimensional DFT is given by:

$$F(k, l) = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} f(i, j) e^{-i2\pi(\frac{ki}{N} + \frac{lj}{N})}$$

Inverse Fourier Transform is given by

$$f(a, b) = \frac{1}{N^2} \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} F(k, l) e^{i2\pi(\frac{ka}{N} + \frac{lb}{N})}$$

3.5 DEEP LEARNING MODELS

3.5.1 Introduction to pre-trained models

A pre-trained model is a model that was trained on a large benchmark dataset to solve a problem similar to the one that we want to solve.

When you're repurposing a pre-trained model for your own needs, you start by removing the original classifier, then you add a new classifier that fits your purposes, and finally you have to fine-tune your model according to one of three strategies:

1. **Train the entire model.** In this case, you use the architecture of the pre-trained model and train it according to your dataset. You're learning the model from scratch, so you'll need a large dataset (and a lot of computational power).
2. **Train some layers and leave the others frozen.** As you remember, lower layers refer to general features (problem independent), while higher layers refer to specific features (problem dependent). Here, we play with that dichotomy by

choosing how much we want to adjust the weights of the network (a frozen layer does not change during training). Usually, if you've a small dataset and a large number of parameters, you'll leave more layers frozen to avoid overfitting. By contrast, if the dataset is large and the number of parameters is small, you can improve your model by training more layers to the new task since overfitting is not an issue.

3. **Freeze the convolutional base.** This case corresponds to an extreme situation of the train/freeze trade-off. The main idea is to keep the convolutional base in its original form and then use its outputs to feed the classifier. You're using the pre-trained model as a fixed feature extraction mechanism, which can be useful if you're short on computational power, your dataset is small, and/or pre-trained model solves a problem very similar to the one you want to solve.

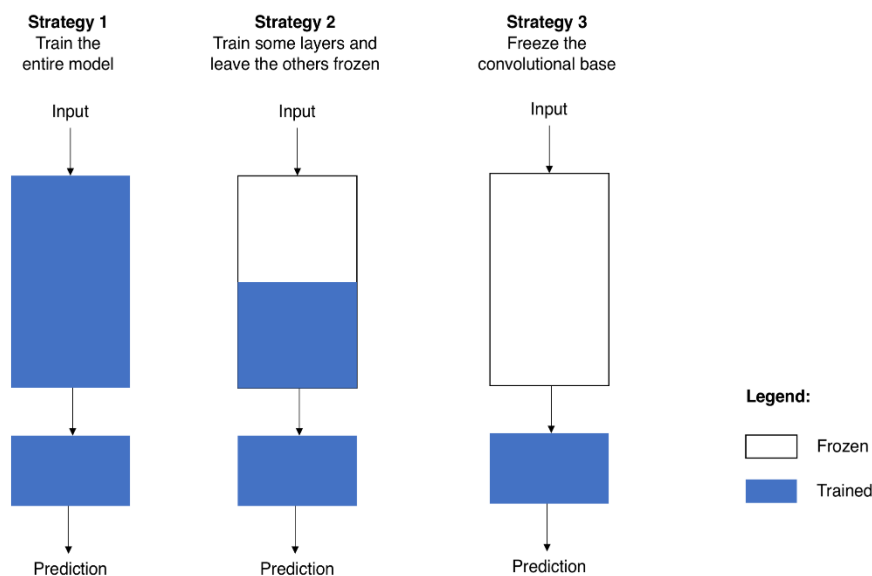


Fig 3.6 Fine tuning Pre- trained models

3.5.2 ResNet152

ResNet owes its name to its residual blocks with skip connections that enable the model to be extremely deep. Even though including skip connections is a common idea in the community now, it was a revolutionary architectural choice and allowed ResNet to reach up to 152 layers with no vanishing or exploding gradient problems during training.

ResNet introduces bypass connections in the network and allows the gradient to flow without getting multiplied with weight matrices several times.

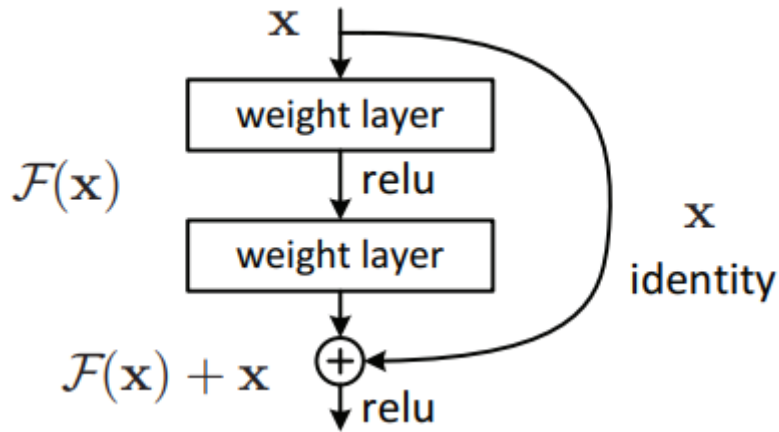


Fig 3.7 Skip Connections

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Fig 3.8 ResNet 152 Architecture

3.5.3 DenseNet121

```
tf.keras.applications.DenseNet121(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
)
```

Arguments

- **include_top**: whether to include the fully-connected layer at the top of the network.
- **weights**: one of None (random initialization), 'imagenet' (pre-training on ImageNet), or the path to the weights file to be loaded.
- **input_tensor**: optional Keras tensor (i.e. output of layers.Input()) to use as image input for the model.
- **input_shape**: optional shape tuple, only to be specified if include_top is False (otherwise the input shape has to be (224, 224, 3) (with 'channels_last' data format) or (3, 224, 224) (with 'channels_first' data format). It should have exactly 3 inputs channels, and width and height should be no smaller than 32. E.g. (200, 200, 3) would be one valid value.
- **pooling**: Optional pooling mode for feature extraction when include_top is False.
 - None means that the output of the model will be the 4D tensor output of the last convolutional block.
 - avg means that global average pooling will be applied to the output of the last convolutional block, and thus the output of the model will be a 2D tensor.
 - max means that global max pooling will be applied.
- **classes**: optional number of classes to classify images into, only to be specified if include_top is True, and if no weights argument is specified.

Returns

A Keras model instance.

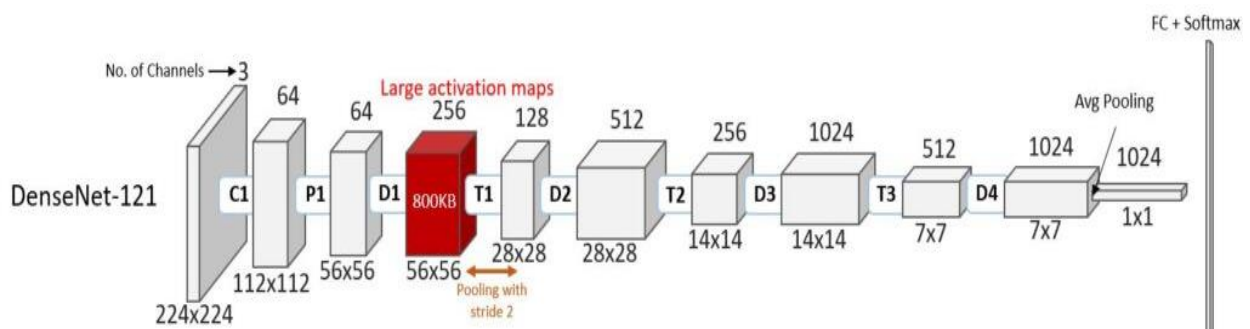


Fig 3.9 DenseNet 121 Architecture

3.5.4 DenseNet169

```
tf.keras.applications.DenseNet169(  
    include_top=True,  
    weights="imagenet",  
    input_tensor=None,  
    input_shape=None,  
    pooling=None,  
    classes=1000,  
)
```

Arguments

- **include_top**: whether to include the fully-connected layer at the top of the network.
- **weights**: one of None (random initialization), 'imagenet' (pre-training on ImageNet), or the path to the weights file to be loaded.
- **input_tensor**: optional Keras tensor (i.e. output of layers.Input()) to use as image input for the model.
- **input_shape**: optional shape tuple, only to be specified if include_top is False (otherwise the input shape has to be (224, 224, 3) (with 'channels_last' data format) or (3, 224, 224) (with 'channels_first' data format). It should have exactly 3 inputs channels, and width and height should be no smaller than 32. E.g. (200, 200, 3) would be one valid value.
- **pooling**: Optional pooling mode for feature extraction when include_top is False.
- **classes**: optional number of classes to classify images into, only to be specified if include_top is True, and if no weights argument is specified.

Layers	Output Size	DenseNet 169
Convolution	112×112	7×7 conv, stride 2
Pooling	56×56	3×3 max pool, stride 2
Dense Block (1)	56×56	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$
Transition Layer (1)	56×56	1×1 conv
	28×28	2×2 average pool, stride 2
Dense Block (2)	28×28	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$
Transition Layer (2)	28×28	1×1 conv
	14×14	2×2 average pool, stride 2
Dense Block (3)	14×14	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$
Transition Layer (3)	14×14	1×1 conv
	7×7	2×2 average pool, stride 2
Dense Block (4)	7×7	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$
Classification Layer	1×1	7×7 global average pool
	1000	1000D fully-connected, softmax

Fig 3.10 DenseNet 169 Architecture

3.6 ENSEMBLE APPROACH

3.6.1 Introduction

Ensemble Learning is all about learning how to best combine predictions from multiple existing models (called the **base-learners**). Every member of the ensemble makes a contribution to the final output and individual weaknesses are offset by the contribution the other members. The combined learned model is named the **meta-learner**. There are several flavors of Ensemble Learning.

- Neural network models are nonlinear and have a high variance, which can be frustrating when preparing a final model for making predictions.
- Ensemble learning combines the predictions from multiple neural network models to reduce the variance of predictions and reduce generalization error.
- Techniques for ensemble learning can be grouped by the element that is varied, such as training data, the model, and how predictions are combined.

It can be helpful to think of varying each of the three major elements of the ensemble method. For example:

- **Training Data:** Vary the choice of data used to train each model in the ensemble.
- **Ensemble Models:** Vary the choice of the models used in the ensemble.
- **Combinations:** Vary the choice of the way that outcomes from ensemble members are combined.

Varying Training Data

The data used to train each member of the ensemble can be varied.

The simplest approach would be to use k-fold cross-validation to estimate the generalization error of the chosen model configuration. In this procedure, k different models are trained on k different subsets of the training data. These k models can then be saved and used as members of an ensemble.

Another popular approach involves resampling the training dataset with replacement, then training a network using the resampled dataset. The resampling procedure means that the composition of each training dataset is different with the possibility of duplicated examples allowing the model trained on the dataset to have a slightly different expectation of the density of the samples, and in turn different generalization error.

This approach is called bootstrap aggregation, or bagging for short, and was designed for use with unpruned decision trees that have high variance and low bias. Typically a large number of decision trees are used, such as hundreds or thousands, given that they are fast to prepare.

Varying Models

Training the same under-constrained model on the same data with different initial conditions will result in different models given the difficulty of the problem, and the stochastic nature of the learning algorithm.

This is because the optimization problem that the network is trying to solve is so challenging that there are many “good” and “different” solutions to map inputs to outputs. This may result in a reduced variance, but may not dramatically improve generalization error. The errors made by the models may still be too highly correlated because the models all have learned similar mapping functions.

An alternative approach might be to vary the configuration of each ensemble model, such as using networks with different capacity (e.g. number of layers or nodes) or models trained under different conditions (e.g. learning rate or regularization).

The result may be an ensemble of models that have learned a more heterogeneous collection of mapping functions and in turn have a lower correlation in their predictions and prediction errors.

Such an ensemble of differently configured models can be achieved through the normal process of developing the network and tuning its hyperparameters. Each model could be saved during this process and a subset of better models chosen to comprise the ensemble.

Varying Combinations

The simplest way to combine the predictions is to calculate the average of the predictions from the ensemble members.

This can be improved slightly by weighting the predictions from each model, where the weights are optimized using a hold-out validation dataset. This provides a weighted average ensemble that is sometimes called model blending.

One further step in complexity involves using a new model to learn how to best combine the predictions from each ensemble member.

The model could be a simple linear model (e.g. much like the weighted average), but could be a sophisticated nonlinear method that also considers the specific input sample in addition to the predictions provided by each member. This general approach of learning a new model is called model stacking, or stacked generalization.

There are more sophisticated methods for stacking models, such as boosting where ensemble members are added one at a time in order to correct the mistakes of prior models. The added complexity means this approach is less often used with large neural network models.

Another combination that is a little bit different is to combine the weights of multiple neural networks with the same structure. The weights of multiple networks can be averaged, to hopefully result in a new single model that has better overall performance than any original model. This approach is called model weight averaging.

3.6.2 Stacking Approach

In stacking, the output of the base-learners are taken as input for training a meta-learner, that learns how to best combine the base-learners predictions.

Often times the stacking model will outperform each of the individual models due to its smoothing nature, offsetting deficiencies of individual models leading to a better prediction performance. Therefore, stacking works best when the base models are basically different.

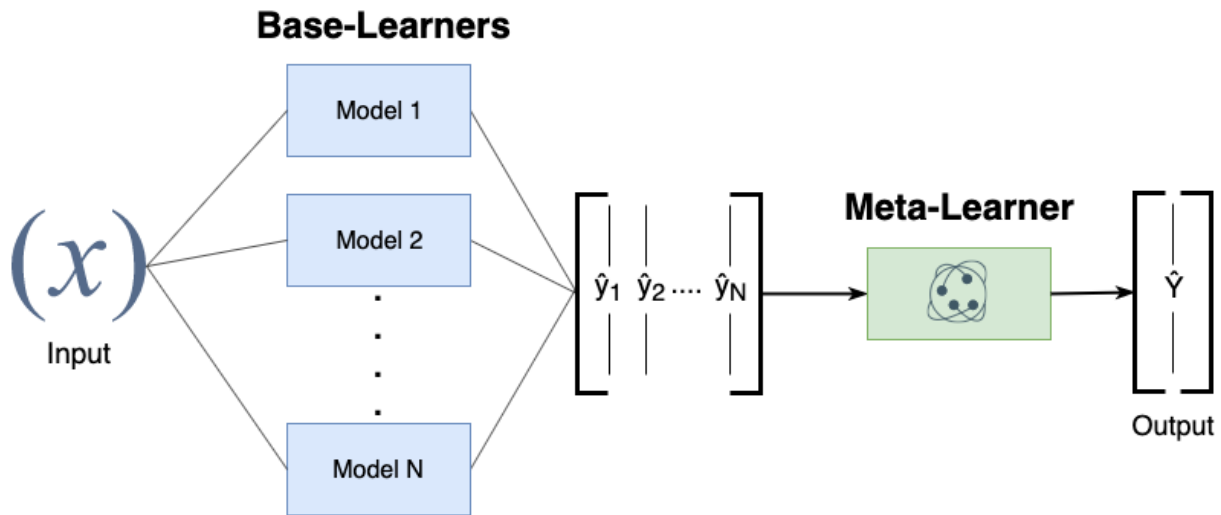


Fig 3.11 Stacking

3.6.3 Weighted Average Ensemble

This method weights the contribution of each ensemble member based on their performance on a hold-out validation dataset. Models with better contribution receive a higher weight.

The main difference between both methods is that in stacking, the meta-learner takes every single output of the base-learners as a training instance, learning how to best map the base-learner decisions into an improved output. The meta-learner can be any classic known machine learning model. The weighted average ensemble on the other hand is just about optimizing weights that are used for weighting all outputs of the base-learner and taking the weighted average. There is no meta-learner here (besides the weights). Here the number of weights is equal to the number of existing base-learners.

$$w_1 \cdot \begin{bmatrix} \hat{y}_1 \\ \vdots \end{bmatrix} + w_2 \cdot \begin{bmatrix} \hat{y}_2 \\ \vdots \end{bmatrix} + \dots + w_n \cdot \begin{bmatrix} \hat{y}_n \\ \vdots \end{bmatrix} = \begin{bmatrix} \hat{Y} \\ \vdots \end{bmatrix}$$

Fig 3.12 Weighted Average Ensemble

3.6.4 Model Averaging Ensemble

Deep learning neural network models are nonlinear methods that learn via a stochastic training algorithm.

This means that they are highly flexible, capable of learning complex relationships between variables and approximating any mapping function, given enough resources. A downside of this flexibility is that the models suffer high variance.

This means that the models are highly dependent on the specific training data used to train the model and on the initial conditions (random initial weights) and serendipity during the training process. The result is a final model that makes different predictions each time the same model configuration is trained on the same dataset.

This can be frustrating when training a final model for use in making predictions on new data, such as operationally or in a machine learning competition.

The high variance of the approach can be addressed by training multiple models for the problem and combining their predictions. This approach is called model averaging and belongs to a family of techniques called ensemble learning.

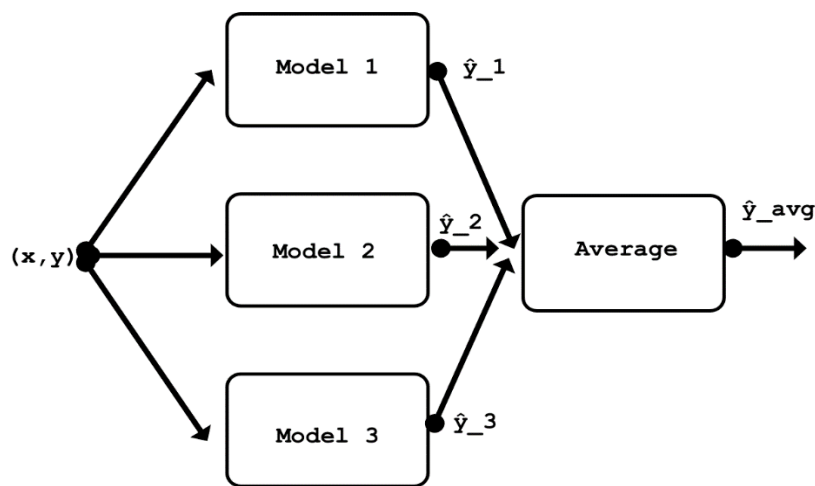


Fig 3.13 Model averaging ensemble

CHAPTER-4

EVALUATION METRICS

4.1 Definition

Evaluation metrics are used to measure the quality of the statistical or machine learning model. Evaluating machine learning models or algorithms is essential for any project. There are many different types of evaluation metrics available to test a model. These include classification accuracy, logarithmic loss, confusion matrix, and others.

4.2 Confusion Matrix

A **confusion matrix**, also known as an error matrix,^[9] is a specific table layout that allows visualization of the performance of an algorithm, typically a supervised learning one (in unsupervised learning it is usually called a **matching matrix**). Each row of the matrix represents the instances in an actual class while each column represents the instances in a predicted class, or vice versa – both variants are found in the literature.^[10] The name stems from the fact that it makes it easy to see whether the system is confusing two classes (i.e. commonly mislabelling one as another).

It is a special kind of contingency table, with two dimensions ("actual" and "predicted"), and identical sets of "classes" in both dimensions (each combination of dimension and class is a variable in the contingency table).

		predicted		
				n
actual		TP	FN	P
		FP	TN	N

Fig 4.1 Confusion Matrix

4.3 Accuracy

Accuracy represents the number of correctly classified data instances over the total number of data instances.

$$accuracy = \frac{TP + TN}{TP + FN + TN + FP}$$

4.4 Precision

Precision should ideally be 1 (high) for a good classifier. Precision becomes 1 only when the numerator and denominator are equal i.e $TP = TP + FP$, this also means FP is zero. As FP increases the value of denominator becomes greater than the numerator and precision value decreases (which we don't want).

$$precision = \frac{TP}{TP + FP}$$

4.5 Recall

Recall is also known as sensitivity or true positive rate and is defined as follows:

$$recall = \frac{TP}{TP + FN}$$

Recall should ideally be 1 (high) for a good classifier. Recall becomes 1 only when the numerator and denominator are equal i.e $TP = TP + FN$, this also means FN is zero. As FN increases the value of denominator becomes greater than the numerator and **recall** value decreases (which we don't want).

4.6 F1- Score

F1-score is a metric which takes into account both precision and recall and is defined as follows:

$$F1 = \frac{2 \times precision \times recall}{precision + recall}$$

F1 Score becomes 1 only when precision and recall are both 1. F1 score becomes high only when both precision and recall are high. F1 score is the harmonic mean of precision and recall and is a better measure than accuracy.

4.7 AUC

The AUC (Area under the curve) of the ROC (Receiver operating characteristic; default) or PR (Precision Recall) curves are quality measures of binary classifiers. Unlike the accuracy, and like cross-entropy losses, ROC-AUC and PR-AUC evaluate all the operational points of a model. This metric creates four local variables, `true_positives`, `true_negatives`, `false_positives` and `false_negatives` that are used to compute the AUC. To discretize the AUC curve, a linearly spaced set of thresholds is used to compute pairs of recall and precision values. The area under the ROC-curve is therefore computed using the height of the recall values by the false positive rate, while the area under the PR-curve is the computed using the height of the precision values by the recall. This value is ultimately returned as `auc`.

CHAPTER-5

SYSTEM DESIGN

5.1 Block Diagram

A block diagram is a specialized, high-level flowchart used in engineering. It is used to design new systems or to describe and improve existing ones. Its structure provides a high-level overview of major system components, key process participants, and important working relationships.

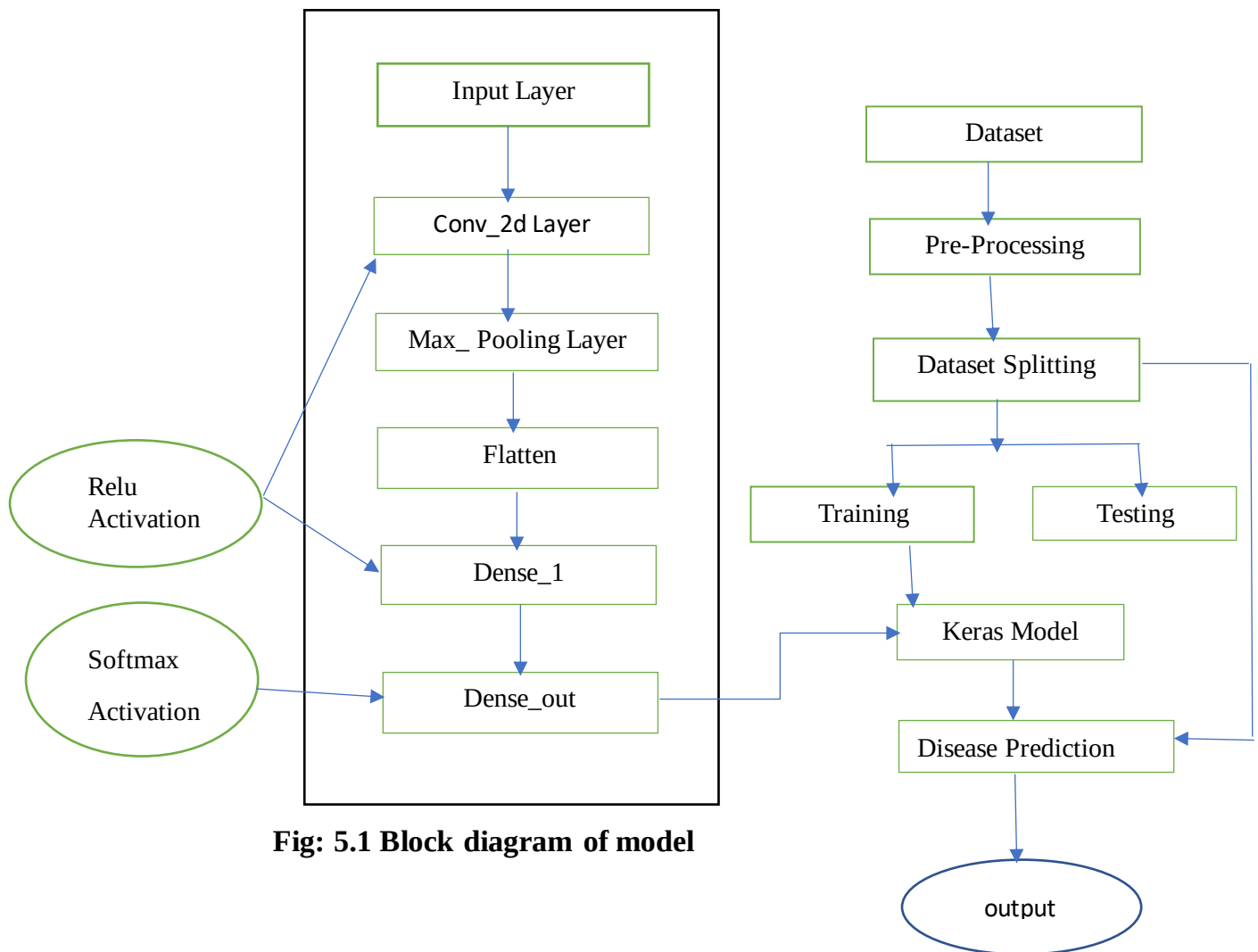


Fig: 5.1 Block diagram of model

5.2 UML Diagrams

5.2.1 Class Diagram

A class diagram in the Unified Modeling Language is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations, and the relationships among objects.

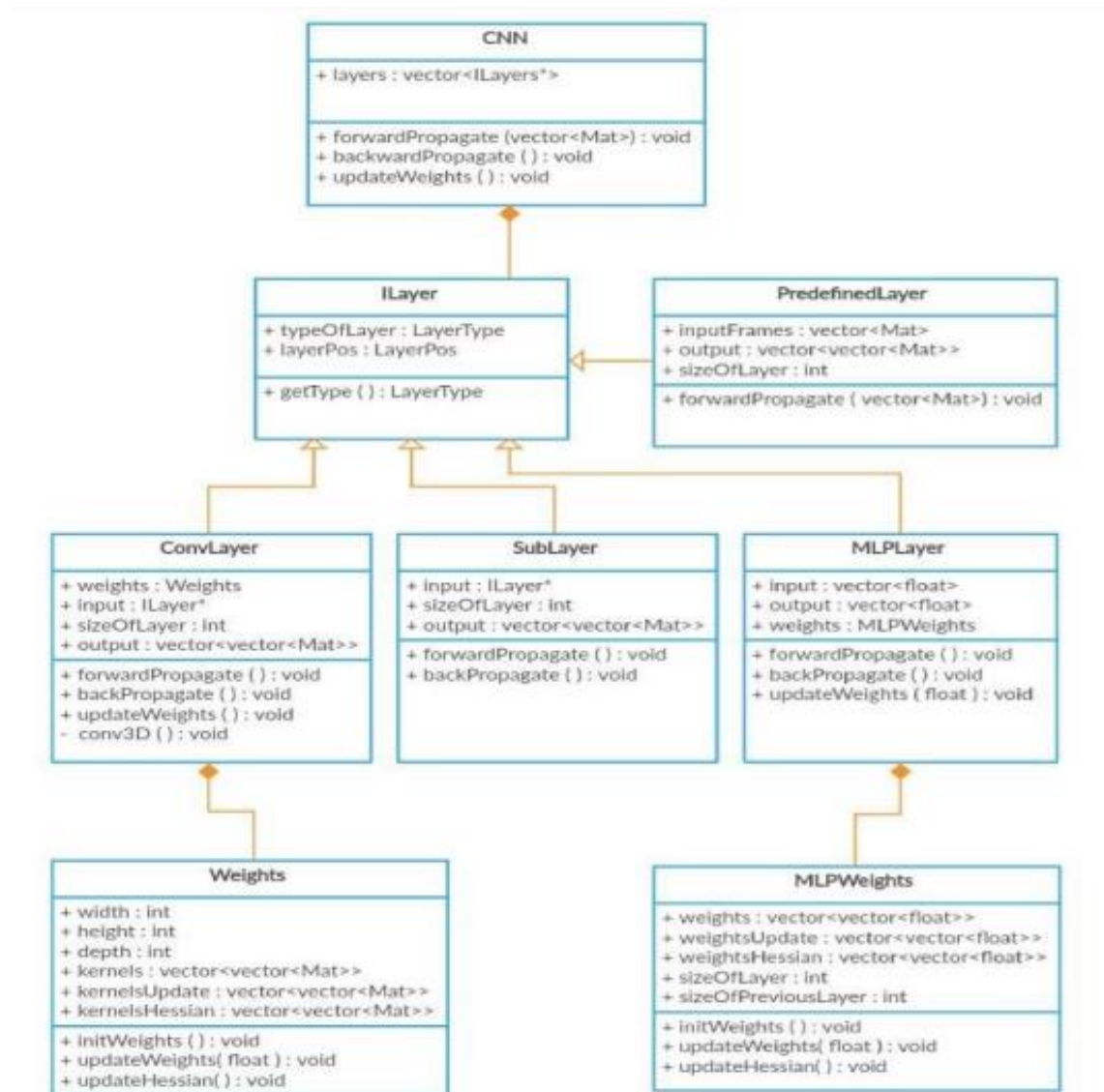


Fig 5.2 Class Diagram

5.2.2 USECASE DIAGRAM

A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved.

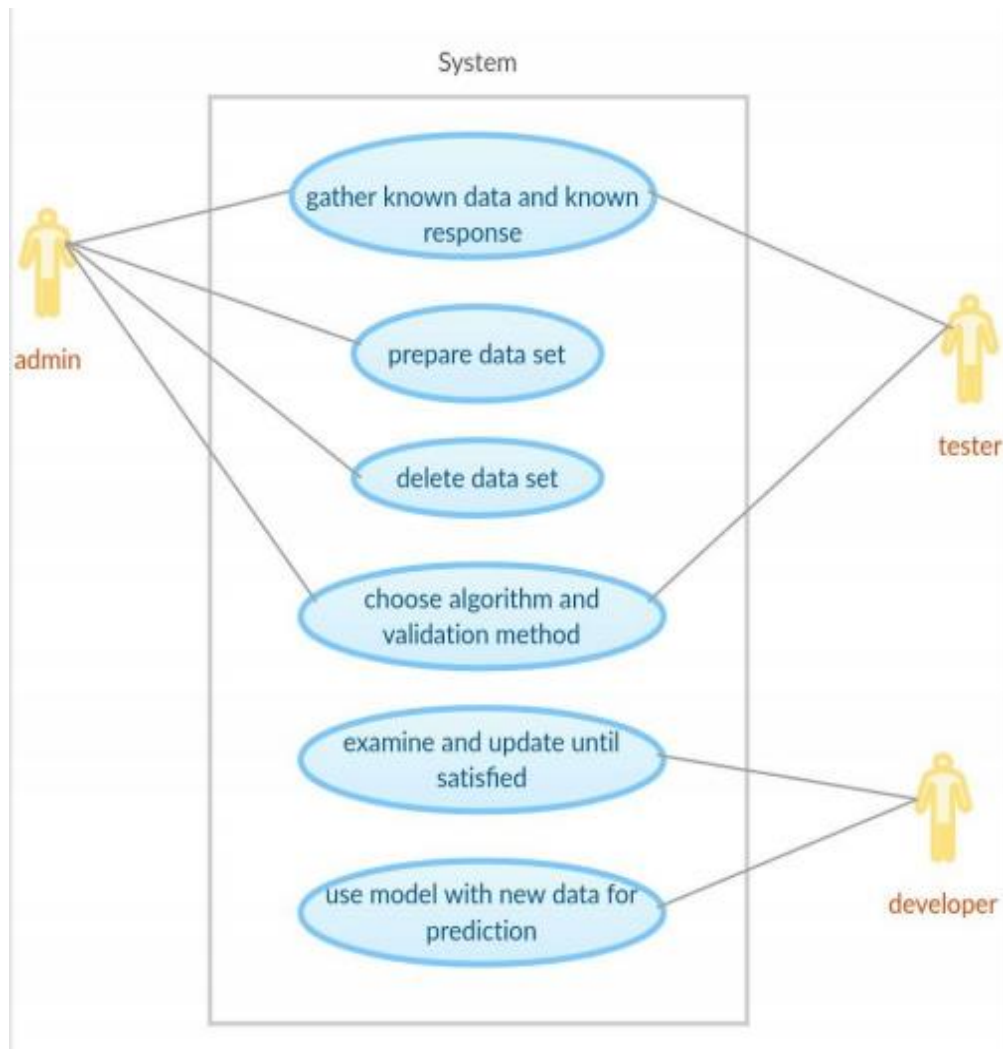


Fig 5.3 Usecase Diagram

5.2.3 SEQUENCE DIAGRAM

A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario.

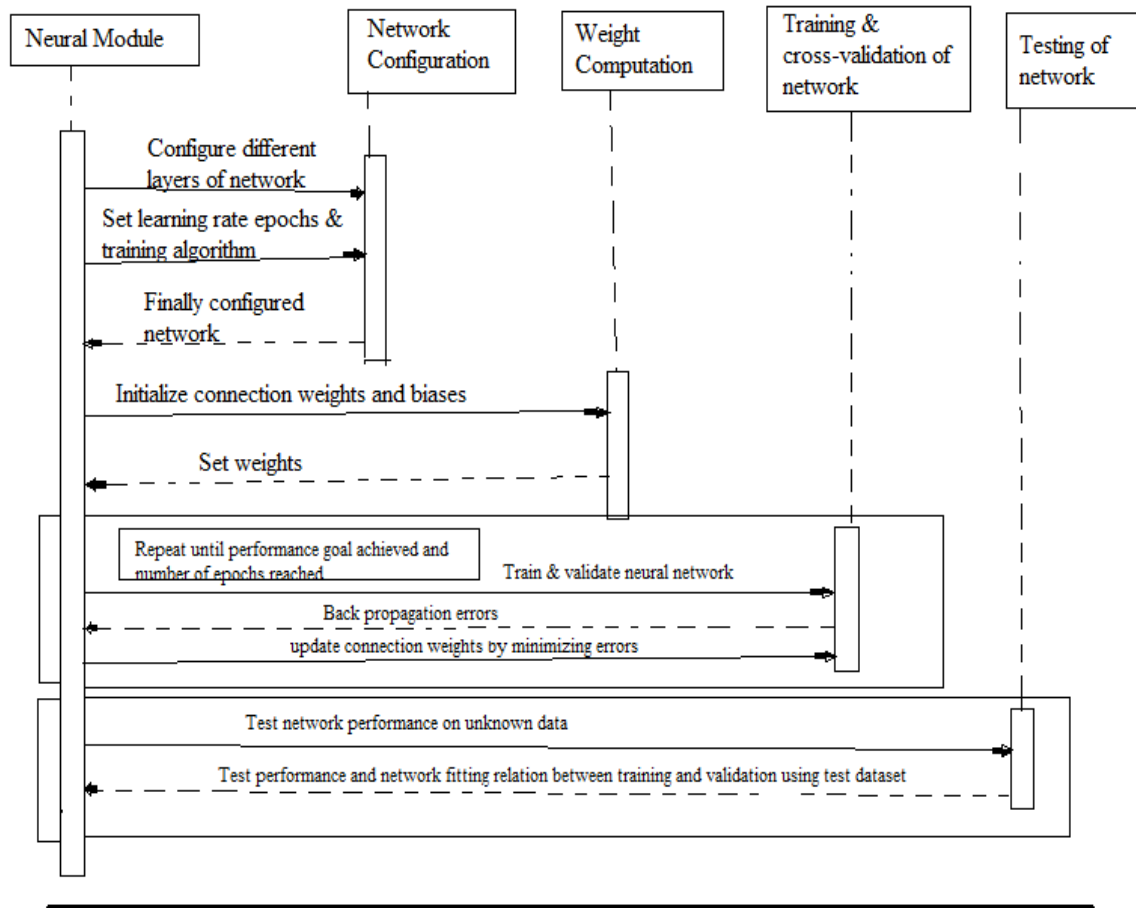


Fig 5.4 Sequence Diagram

5.2.4 STATE CHART DIAGRAM

A state diagram is a type of diagram used in computer science and related fields to describe the behaviour of systems. State diagrams require that the system described is composed of a finite number of states; sometimes, this is indeed the case, while at other times this is a reasonable abstraction.

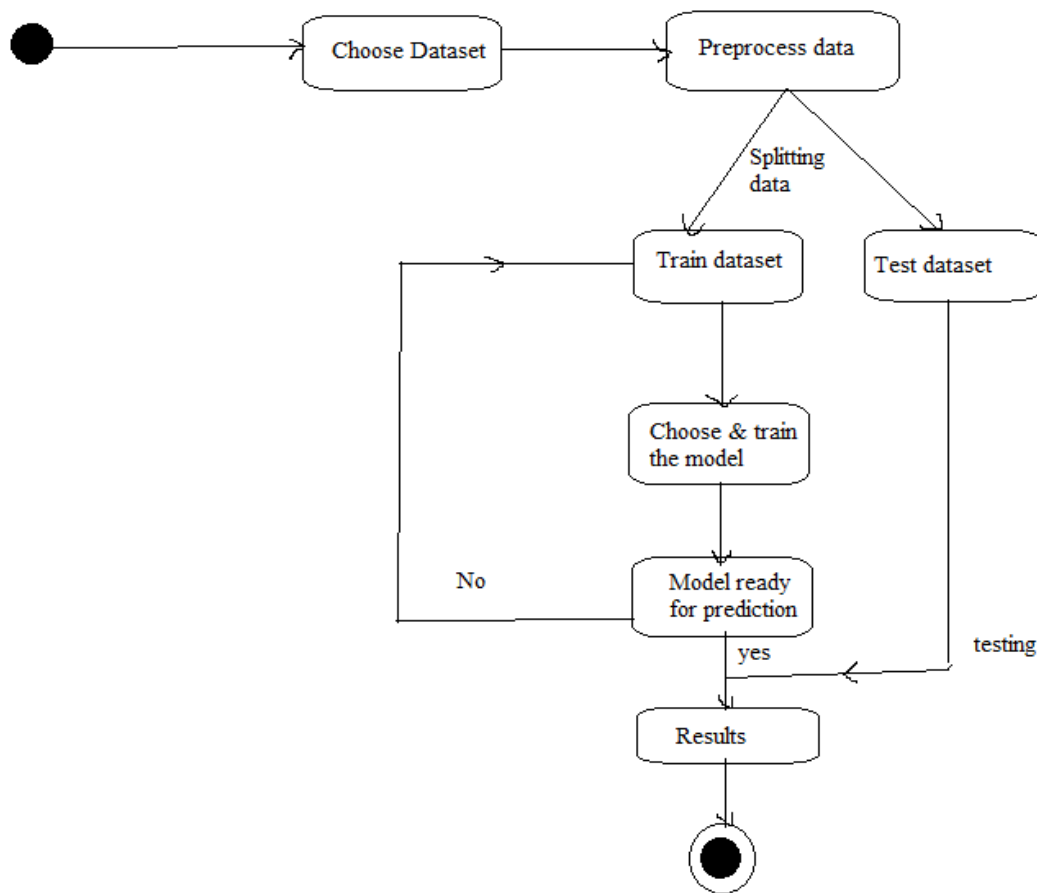


Fig 5.5 State Chart Diagram

5.2.5 ACTIVITY DIAGRAM

Activity diagram is another important diagram in UML to describe the dynamic aspects of the system. Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.

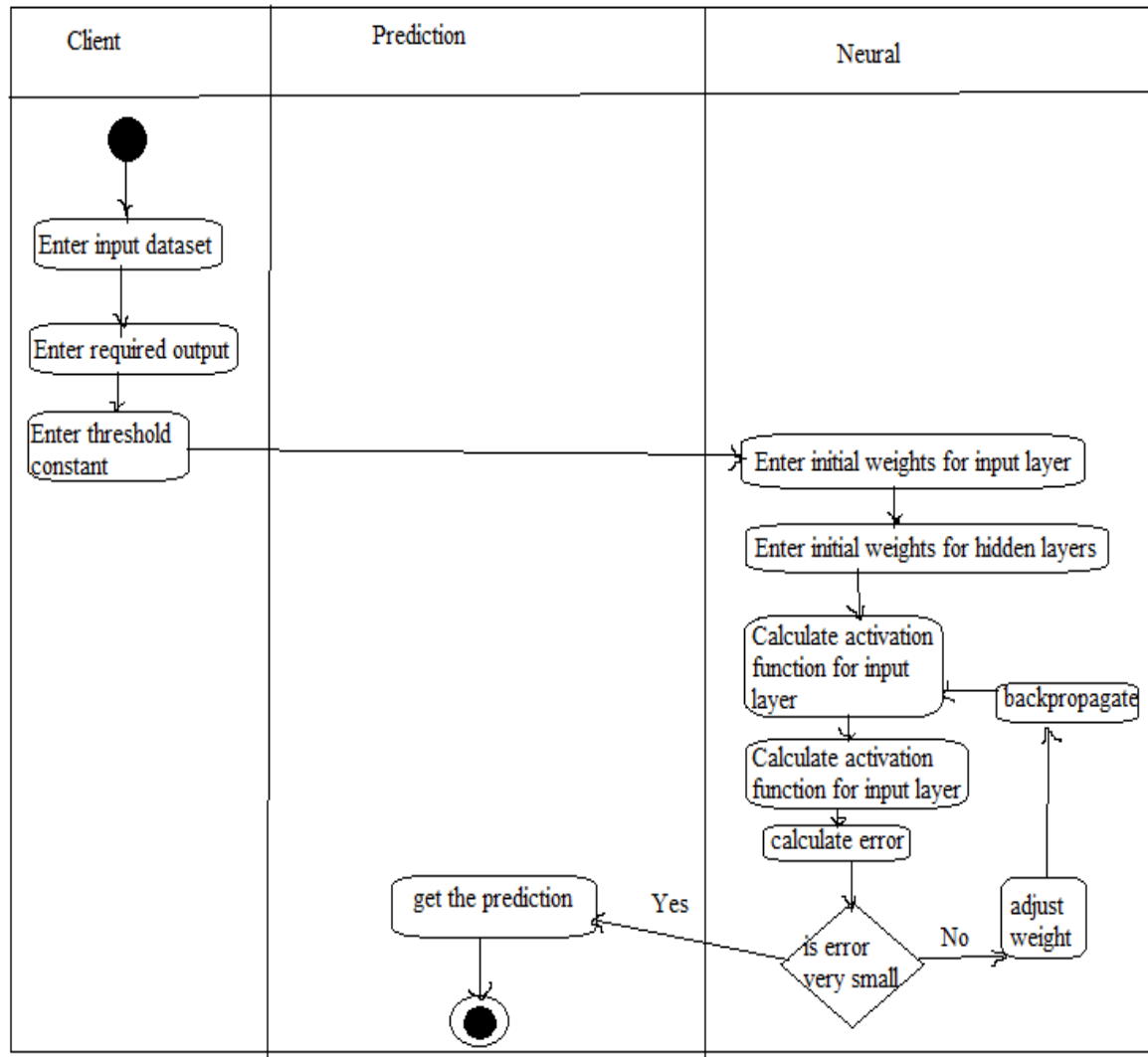


Fig 5.6 Activity Diagram

5.2.6 OBJECT DIAGRAM

Object diagrams are derived from class diagrams so object diagrams are dependent upon class diagrams. Object diagrams represent an instance of a class diagram. The basic concepts are similar for class diagrams and object diagrams. Object diagrams also represent the static view of a system but this static view is a snapshot of the system at a particular moment.

Object diagrams are used to render a set of objects and their relationships as an instance. The purposes of object diagrams are similar to class diagrams. The difference is that a class diagram represents an abstract model consisting of classes and their relationships. However, an object diagram represents an instance at a particular moment, which is concrete in nature.

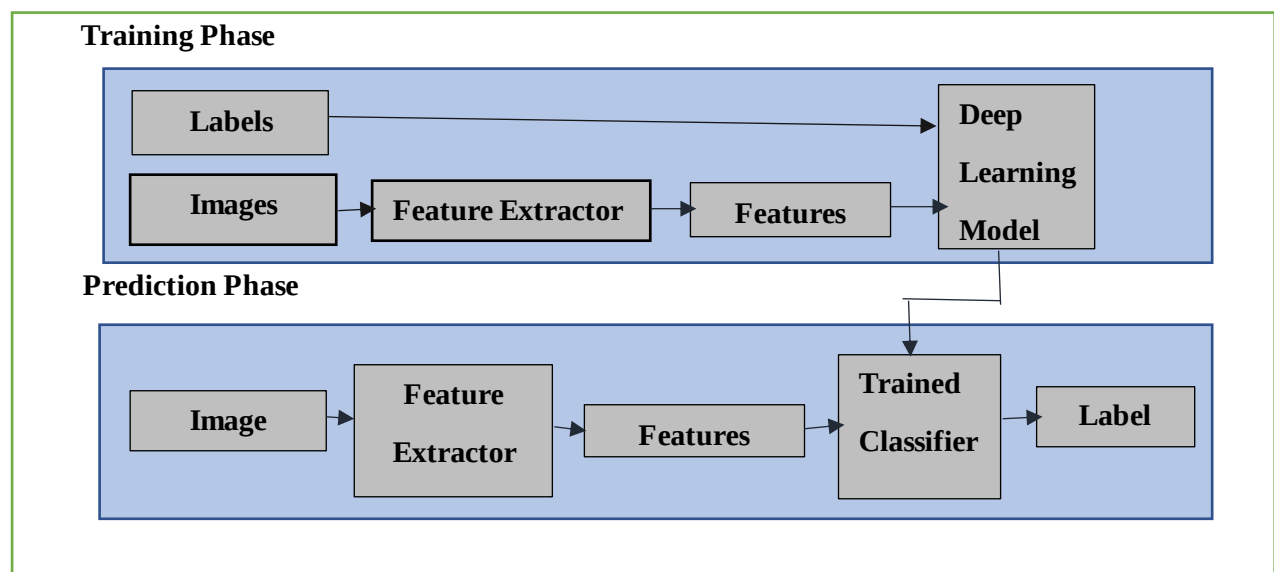


Fig 5.7 Object Diagram

CHAPTER-6

TESTING

Software Testing is a process of executing the application with intent to find any software bugs. It is used to check whether the application met its expectations and all the functionalities of the application are working. The final goal of testing is to check whether the application is behaving in the way it is supposed to under specified conditions. All aspects of the code are examined to check the quality of the application. The primary purpose of testing is to detect software failures so that defects may be uncovered and corrected. The test cases are designed in such way that scope of finding the bugs is maximum.

6.1 TESTING LEVELS

There are various testing levels based on the specificity of the test.

Unit Testing: - Unit testing refers to tests conducted on a section of code in order to verify the functionality of that piece of code. This is done at the function level.

Integration Testing: - Integration testing is any type of software testing that seeks to verify the interfaces between components of a software design. Its primary purpose is to expose the defects associated with the interfacing of modules.

System Testing: - System testing tests a completely integrated system to verify that the system meets its requirements.

Acceptance Testing: - Acceptance testing tests the readiness of application, satisfying all requirements.

Performance Testing: - Performance testing is the process of determining the speed or effectiveness of a computer, network, software program or devices such as response time or millions of instructions per second etc.

6.2 SYSTEM TEST CASES

A test case is a set of test data, preconditions, expected results and post conditions, developed for a test scenario to verify compliance with a specific requirement. I have designed and executed a few test cases to check if the project meets the functional requirements.

TEST CONDITION	INPUT SPECIFICATION	OUTPUT SPECIFICATION	PASS/ FAIL
Testing with the images in project dataset	Fundus Image of Retina	Predication label of input image and accuracy	PASS

Table 6.1 Test case for images present in dataset

TEST CONDITION	INPUT SPECIFICATION	OUTPUT SPECIFICATION	PASS/ FAIL
Testing with the other images	Fundus Image of Retina not in dataset	Stage of DR	PASS

Table 6.2 Test case for images not present in dataset

The testers need to focus on for the following:-

- Test with new data, rather than the original training data. If necessary, split your training set into two groups: one that does training, and one that does testing. Better, obtain and use fresh data if you are able.
- Understand the architecture of the network as a part of the testing process. Testers won't necessarily understand how the neural network was constructed, but need to understand whether it meets requirements. And based on the measurements that they are testing, they may have to recommend a radically different approach, or admit the software is just not capable of doing what it was asked to do with confidence.

CHAPTER-7

IMPLEMENTATION

//CODE FOR PREDICTION OF DR USING ENTIRE DATASET

//train.csv file consists of 3662 samples belonging to 5 levels

// 0- 1805 1- 370 2- 999 3- 193 4- 295

//Importing required packages

```
import os
import warnings
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, cohen_kappa_score
from keras.models import Model
from keras import optimizers, applications
from keras.preprocessing.image import ImageDataGenerator
from keras.callbacks import EarlyStopping, ReduceLROnPlateau
from keras.layers import Dense, Dropout, GlobalAveragePooling2D, Input
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image_dataset_from_directory
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import InputLayer, BatchNormalization, Dropout, Flatten, Dense, Activation, MaxPool2D, Conv2D
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.utils import to_categorical
from keras import optimizers
from tensorflow.keras.optimizers import Adam
from keras.callbacks import Callback, ModelCheckpoint, ReduceLROnPlateau
from keras.models import Sequential, load_model
from keras.layers import Dense, Dropout
from keras.wrappers.scikit_learn import KerasClassifier
```

```
import keras.backend as K
from typing import Optional
warnings.filterwarnings("ignore")
```

//Reading csv file using pandas

```
train = pd.read_csv(r'C:\Users\DELL\Desktop\APTOS_DR\aptos2019-blindness-
detection/train.csv')
```

//Printing number of samples and first 5 samples in CSV file

```
print('Number of train samples: ', train.shape[0])
display(train.head())
```

```
Number of train samples: 3662
```

	id_code	diagnosis
0	000c1434d8d7	2
1	001639a390f0	4
2	0024cdab0c1e	1
3	002c21358ce6	0
4	005b95c28852	0

//Returns number of unique classes in diagnosis column in csv file

```
N_CLASSES = train['diagnosis'].nunique()
N_CLASSES
```

// Formatting image name and converting numeric type of diagnosis to string

```
train["id_code"] = train["id_code"].apply(lambda x: x + ".png")
train['diagnosis'] = train['diagnosis'].astype('str')
train.head()
```

//Creating Image Data Generator object

```
train_datagen=ImageDataGenerator(rescale=1./255,
                                  validation_split=0.2,
                                  horizontal_flip=True)
```

//Creating train and test generator based on validation split specified

//Train Generator

```

train_generator=train_datagen.flow_from_dataframe(
    dataframe=train,
    directory=r"C:\Users\DELL\Desktop\APTOS_DR\aptos2019-blindness
detection/train_images/",
    x_col="id_code",
    y_col="diagnosis",
    batch_size=16,
    class_mode="categorical",
    target_size=(224, 224),
    subset='training')

```

//Validation generator

```

valid_generator=train_datagen.flow_from_dataframe(
    dataframe=train,
    directory=r"C:\Users\DELL\Desktop\APTOS_DR\aptos2019-blindness-
detection/train_images/",
    x_col="id_code",
    y_col="diagnosis",
    batch_size=16,
    class_mode="categorical",
    target_size=(224, 224),
    subset='validation')

```

//DenseNet169 Model

```

from keras.applications.densenet import DenseNet169

base_model = DenseNet169(input_shape=(224, 224, 3), include_top=False,
weights='imagenet')

```

```

Epoch 1/10
183/183 [=====] - 2959s 16s/step - loss: 0.9213 - accuracy: 0.6709 - precision: 0.8572 - recall: 0.504
8 - auc: 0.8911 - val_loss: 0.6595 - val_accuracy: 0.7736 - val_precision: 0.8548 - val_recall: 0.6625 - val_auc: 0.9479
Epoch 2/10
183/183 [=====] - 2999s 16s/step - loss: 0.7279 - accuracy: 0.7426 - precision: 0.8270 - recall: 0.631
4 - auc: 0.9343 - val_loss: 0.6010 - val_accuracy: 0.7833 - val_precision: 0.8409 - val_recall: 0.6972 - val_auc: 0.9562
Epoch 3/10
183/183 [=====] - 2881s 16s/step - loss: 0.6504 - accuracy: 0.7618 - precision: 0.8384 - recall: 0.669
5 - auc: 0.9479 - val_loss: 0.5784 - val_accuracy: 0.7847 - val_precision: 0.8540 - val_recall: 0.7069 - val_auc: 0.9578
Epoch 4/10
183/183 [=====] - 2819s 15s/step - loss: 0.6201 - accuracy: 0.7721 - precision: 0.8494 - recall: 0.683
3 - auc: 0.9522 - val_loss: 0.5635 - val_accuracy: 0.7917 - val_precision: 0.8661 - val_recall: 0.6917 - val_auc: 0.9601
Epoch 5/10
183/183 [=====] - 2929s 16s/step - loss: 0.5686 - accuracy: 0.7896 - precision: 0.8631 - recall: 0.714
1 - auc: 0.9593 - val_loss: 0.5545 - val_accuracy: 0.7944 - val_precision: 0.8506 - val_recall: 0.7431 - val_auc: 0.9607
Epoch 6/10
183/183 [=====] - 4193s 23s/step - loss: 0.5340 - accuracy: 0.8047 - precision: 0.8588 - recall: 0.741
2 - auc: 0.9640 - val_loss: 0.5537 - val_accuracy: 0.7931 - val_precision: 0.8731 - val_recall: 0.7264 - val_auc: 0.9612
Epoch 7/10
183/183 [=====] - 2872s 16s/step - loss: 0.5159 - accuracy: 0.8085 - precision: 0.8652 - recall: 0.749
1 - auc: 0.9663 - val_loss: 0.5403 - val_accuracy: 0.8042 - val_precision: 0.8395 - val_recall: 0.7556 - val_auc: 0.9628
Epoch 8/10
183/183 [=====] - 2846s 16s/step - loss: 0.5251 - accuracy: 0.7986 - precision: 0.8511 - recall: 0.747
4 - auc: 0.9651 - val_loss: 0.5305 - val_accuracy: 0.8069 - val_precision: 0.8502 - val_recall: 0.7806 - val_auc: 0.9639
Epoch 9/10
183/183 [=====] - 2251s 12s/step - loss: 0.4841 - accuracy: 0.8222 - precision: 0.8751 - recall: 0.772
1 - auc: 0.9699 - val_loss: 0.5250 - val_accuracy: 0.8083 - val_precision: 0.8477 - val_recall: 0.7806 - val_auc: 0.9645
Epoch 10/10
183/183 [=====] - 1697s 9s/step - loss: 0.4655 - accuracy: 0.8305 - precision: 0.8685 - recall: 0.7793
- auc: 0.9725 - val_loss: 0.5448 - val_accuracy: 0.8069 - val_precision: 0.8528 - val_recall: 0.7722 - val_auc: 0.9620

```

//DenseNet121 Model

from keras.applications.densenet import DenseNet121

base_model = DenseNet121(input_shape=(224, 224, 3), include_top=False, weights='imagenet')

```
Epoch 1/10
183/183 [=====] - 2953s 16s/step - loss: 0.9581 - accuracy: 0.6658 - precision: 0.8038 - recall: 0.503
4 - auc: 0.8822 - val_loss: 0.6803 - val_accuracy: 0.7569 - val_precision: 0.8261 - val_recall: 0.6597 - val_auc: 0.9414
Epoch 2/10
183/183 [=====] - 3011s 16s/step - loss: 0.7215 - accuracy: 0.7382 - precision: 0.8385 - recall: 0.638
0 - auc: 0.9352 - val_loss: 0.6759 - val_accuracy: 0.7681 - val_precision: 0.8750 - val_recall: 0.6125 - val_auc: 0.9430
Epoch 3/10
183/183 [=====] - 2873s 16s/step - loss: 0.6876 - accuracy: 0.7491 - precision: 0.8338 - recall: 0.654
4 - auc: 0.9408 - val_loss: 0.5920 - val_accuracy: 0.7847 - val_precision: 0.8557 - val_recall: 0.7083 - val_auc: 0.9562
Epoch 4/10
183/183 [=====] - 2818s 15s/step - loss: 0.6257 - accuracy: 0.7667 - precision: 0.8398 - recall: 0.685
8 - auc: 0.9511 - val_loss: 0.5775 - val_accuracy: 0.7861 - val_precision: 0.8438 - val_recall: 0.7125 - val_auc: 0.9578
Epoch 5/10
183/183 [=====] - 2952s 16s/step - loss: 0.6142 - accuracy: 0.7721 - precision: 0.8441 - recall: 0.692
9 - auc: 0.9526 - val_loss: 0.5632 - val_accuracy: 0.7958 - val_precision: 0.8409 - val_recall: 0.7486 - val_auc: 0.9594
Epoch 6/10
183/183 [=====] - 4223s 23s/step - loss: 0.5995 - accuracy: 0.7797 - precision: 0.8412 - recall: 0.714
5 - auc: 0.9554 - val_loss: 0.5672 - val_accuracy: 0.8014 - val_precision: 0.8469 - val_recall: 0.7375 - val_auc: 0.9591
Epoch 7/10
183/183 [=====] - ETA: 0s - loss: 0.5685 - accuracy: 0.7944 - precision: 0.8523 - recall: 0.7189 - au
c: 0.9591
Epoch 00007: ReduceLROnPlateau reducing learning rate to 0.000800000037997961.
183/183 [=====] - 2867s 16s/step - loss: 0.5685 - accuracy: 0.7944 - precision: 0.8523 - recall: 0.718
9 - auc: 0.9591 - val_loss: 0.5716 - val_accuracy: 0.7931 - val_precision: 0.8418 - val_recall: 0.7389 - val_auc: 0.9584
Epoch 00007: early stopping
```

//ResNet152 Model

base_model =

tf.keras.applications.ResNet152V2(input_shape=(224,224,3),include_top=False,weights="imagenet")

```
Epoch 1/10
183/183 [=====] - 3535s 19s/step - loss: 1.1165 - accuracy: 0.5899 - precision: 0.7712 - recall: 0.400
1 - auc: 0.8374 - val_loss: 1.0588 - val_accuracy: 0.7222 - val_precision: 0.7470 - val_recall: 0.6931 - val_auc: 0.9106
Epoch 2/10
183/183 [=====] - 3024s 17s/step - loss: 0.8077 - accuracy: 0.7189 - precision: 0.8222 - recall: 0.604
7 - auc: 0.9175 - val_loss: 0.7750 - val_accuracy: 0.7264 - val_precision: 0.7941 - val_recall: 0.6750 - val_auc: 0.9242
Epoch 3/10
183/183 [=====] - 3076s 17s/step - loss: 0.7346 - accuracy: 0.7310 - precision: 0.8190 - recall: 0.629
0 - auc: 0.9319 - val_loss: 0.8083 - val_accuracy: 0.7250 - val_precision: 0.7788 - val_recall: 0.6750 - val_auc: 0.9203
Epoch 4/10
183/183 [=====] - ETA: 0s - loss: 0.7340 - accuracy: 0.7286 - precision: 0.8084 - recall: 0.6400 - au
c: 0.9317
Epoch 00004: ReduceLROnPlateau reducing learning rate to 0.000800000037997961.
183/183 [=====] - 2997s 16s/step - loss: 0.7340 - accuracy: 0.7286 - precision: 0.8084 - recall: 0.640
0 - auc: 0.9317 - val_loss: 0.7750 - val_accuracy: 0.7458 - val_precision: 0.7828 - val_recall: 0.6958 - val_auc: 0.9269
Epoch 5/10
183/183 [=====] - 3137s 17s/step - loss: 0.7172 - accuracy: 0.7409 - precision: 0.8192 - recall: 0.648
2 - auc: 0.9351 - val_loss: 0.6725 - val_accuracy: 0.7500 - val_precision: 0.8107 - val_recall: 0.6722 - val_auc: 0.9423
Epoch 6/10
183/183 [=====] - 4528s 25s/step - loss: 0.6774 - accuracy: 0.7546 - precision: 0.8263 - recall: 0.674
0 - auc: 0.9423 - val_loss: 0.6690 - val_accuracy: 0.7611 - val_precision: 0.8208 - val_recall: 0.6681 - val_auc: 0.9437
Epoch 7/10
183/183 [=====] - 3097s 17s/step - loss: 0.6558 - accuracy: 0.7581 - precision: 0.8288 - recall: 0.676
0 - auc: 0.9457 - val_loss: 0.9118 - val_accuracy: 0.7403 - val_precision: 0.7608 - val_recall: 0.7069 - val_auc: 0.9218
Epoch 8/10
183/183 [=====] - ETA: 0s - loss: 0.6746 - accuracy: 0.7594 - precision: 0.8252 - recall: 0.6771 - au
c: 0.9425
Epoch 00008: ReduceLROnPlateau reducing learning rate to 0.0006400000303983689.
183/183 [=====] - 2554s 14s/step - loss: 0.6746 - accuracy: 0.7594 - precision: 0.8252 - recall: 0.677
1 - auc: 0.9425 - val_loss: 0.6833 - val_accuracy: 0.7542 - val_precision: 0.8088 - val_recall: 0.6931 - val_auc: 0.9423
```


//Tuning pre-trained models

```
for layer in base_model.layers[:-10]:  
    layer.trainable=False
```

//Adding tuned pre-trained model to Sequential model

```
model=Sequential()  
model.add(base_model)  
model.add(Dropout(0.5))  
model.add(Flatten())  
model.add(BatchNormalization())  
model.add(Dense(256,kernel_initializer='he_uniform'))  
model.add(BatchNormalization())  
model.add(Activation('relu'))  
model.add(Dropout(0.5))  
model.add(Dense(128,kernel_initializer='he_uniform'))  
model.add(BatchNormalization())  
model.add(Activation('relu'))  
model.add(Dropout(0.5))  
model.add(Dense(32,kernel_initializer='he_uniform'))  
model.add(BatchNormalization())  
model.add(Activation('relu'))  
model.add(Dense(5,activation='softmax'))
```

//Defining Metrics

```
METRICS = [  
    tf.keras.metrics.CategoricalAccuracy(name='accuracy'),  
    tf.keras.metrics.Precision(name='precision'),  
    tf.keras.metrics.Recall(name='recall'),  
    tf.keras.metrics.AUC(name='auc'),  
]
```

//Defining Callbacks

```
lrd = ReduceLROnPlateau(monitor = 'val_loss',patience = 2,verbose = 1,factor = 0.8,  
min_lr = 1e-6)
```

```
mcp = ModelCheckpoint('DenseNet169.h5')
```

```
es = EarlyStopping(verbose=1, patience=2)
```

//Compiling the model

```
model.compile(optimizer='Adam', loss="categorical_crossentropy",  
metrics=METRICS)
```

//Calculating the step sizes for train and valid generators

```
STEP_SIZE_TRAIN = train_generator.n//train_generator.batch_size
```

```
STEP_SIZE_VALID = valid_generator.n//valid_generator.batch_size
```

//Running the model

```
%time
```

```
history =
```

```
model.fit_generator(generator=train_generator,steps_per_epoch=STEP_SIZE_TRAIN  
,validation_data=valid_generator,validation_steps=STEP_SIZE_VALID,epochs=10,c  
allbacks=[lrd,mcp,es])
```

//CODE FOR PREDICTION OF DR USING ENTIRE DATASET

```
// 0- No DR 1-DR
```

//Reading csv file using pandas

```
train = pd.read_csv(r'C:\Users\DELL\Desktop\APTOS_DR\aptos2019-blindness-  
detection/train.csv')
```

//Merging 1,2,3,4 levels of DR to single level

```
train=train.replace(to_replace=[2,3,4],value=1)
```

//Printing number of samples and first 5 samples in CSV file

```
print('Number of train samples: ', train.shape[0])
display(train.head())
```

```
//Returns number of unique classes in diagnosis column in csv file
```

```
N_CLASSES = train['diagnosis'].nunique()
N_CLASSES
```

```
// Formatting image name and converting numeric type of diagnosis to string
```

```
train["id_code"] = train["id_code"].apply(lambda x: x + ".png")
train['diagnosis'] = train['diagnosis'].astype('str')
train.head()
```

```
//Creating Image Data Generator object
```

```
train_datagen=ImageDataGenerator(rescale=1./255,
                                  validation_split=0.2,
                                  horizontal_flip=True)
```

```
//Creating train and test generator based on validation split specified
```

```
//Train Generator
```

```
train_generator=train_datagen.flow_from_dataframe(
    dataframe=train,
    directory=r"C:\Users\DELL\Desktop\APTOS_DR\aptos2019-blindness-
detection/train_images/",
    x_col="id_code",
    y_col="diagnosis",
    batch_size=16,
    class_mode="categorical",
    target_size=(224, 224),
    subset='training')
```

```
//Validation generator
```

```
valid_generator=train_datagen.flow_from_dataframe(
    dataframe=train,
    directory=r"C:\Users\DELL\Desktop\APTOS_DR\aptos2019-blindness-
detection/train_images/",
```

```

x_col="id_code",
y_col="diagnosis",
batch_size=16,
class_mode="categorical",
target_size=(224, 224),
subset='validation')

```

//DenseNet169 Model

```

from keras.applications.densenet import DenseNet169

base_model = DenseNet169(input_shape=(224, 224, 3), include_top=False,
weights='imagenet')

```

```

Epoch 1/2
183/183 [=====] - 892s 5s/step - loss: 0.2606 - accuracy: 0.9087 - precision: 0.9369 - recall: 0.8764
- auc: 0.9626 - val_loss: 0.0999 - val_accuracy: 0.9694 - val_precision: 0.9763 - val_recall: 0.9661 - val_auc: 0.9925
Epoch 2/2
183/183 [=====] - 939s 5s/step - loss: 0.1553 - accuracy: 0.9499 - precision: 0.9555 - recall: 0.9437
- auc: 0.9841 - val_loss: 0.0899 - val_accuracy: 0.9708 - val_precision: 0.9868 - val_recall: 0.9590 - val_auc: 0.9956

```

//DenseNet121 Model

```

from keras.applications.densenet import DenseNet121

base_model = DenseNet121(input_shape=(224, 224, 3), include_top=False,
weights='imagenet')

```

```

Epoch 1/2
183/183 [=====] - 841s 5s/step - loss: 0.2606 - accuracy: 0.9146 - precision: 0.9187 - recall: 0.9092
- auc: 0.9644 - val_loss: 0.1246 - val_accuracy: 0.9667 - val_precision: 0.9865 - val_recall: 0.9506 - val_auc: 0.9867
Epoch 2/2
183/183 [=====] - 949s 5s/step - loss: 0.1717 - accuracy: 0.9478 - precision: 0.9552 - recall: 0.9394
- auc: 0.9800 - val_loss: 0.1052 - val_accuracy: 0.9653 - val_precision: 0.9841 - val_recall: 0.9513 - val_auc: 0.9929

```

//ResNet152 Model

```

base_model =
tf.keras.applications.ResNet152V2(input_shape=(224,224,3),include_top=False,weigh
ts="imagenet")

```

```

Epoch 1/2
183/183 [=====] - 1143s 6s/step - loss: 0.2830 - accuracy: 0.9077 - precision: 0.9121 - recall: 0.9014
- auc: 0.9579 - val_loss: 0.9174 - val_accuracy: 0.8306 - val_precision: 0.9855 - val_recall: 0.6974 - val_auc: 0.9263
Epoch 2/2
183/183 [=====] - 1209s 7s/step - loss: 0.1874 - accuracy: 0.9410 - precision: 0.9476 - recall: 0.9332
- auc: 0.9762 - val_loss: 0.2855 - val_accuracy: 0.9292 - val_precision: 0.9746 - val_recall: 0.8918 - val_auc: 0.9724

```

//Tuning pre-trained models

```
for layer in base_model.layers[:-10]:
```

```
    layer.trainable=False
```

//Adding tuned pre-trained model to Sequential model

```
model=Sequential()
```

```
model.add(base_model)
```

```
model.add(Dropout(0.5))
```

```
model.add(Flatten())
```

```
model.add(BatchNormalization())
```

```
model.add(Dense(256,kernel_initializer='he_uniform'))
```

```
model.add(BatchNormalization())
```

```
model.add(Activation('relu'))
```

```
model.add(Dropout(0.5))
```

```
model.add(Dense(128,kernel_initializer='he_uniform'))
```

```
model.add(BatchNormalization())
```

```
model.add(Activation('relu'))
```

```
model.add(Dropout(0.5))
```

```
model.add(Dense(32,kernel_initializer='he_uniform'))
```

```
model.add(BatchNormalization())
```

```
model.add(Activation('relu'))
```

```
model.add(Dense(5,activation='softmax'))
```

//Defining Metrics

```
METRICS = [
```

```
    tf.keras.metrics.CategoricalAccuracy(name='accuracy'),
```

```
    tf.keras.metrics.Precision(name='precision'),
```

```
    tf.keras.metrics.Recall(name='recall'),
```

```
    tf.keras.metrics.AUC(name='auc'),
```

```
]
```

//Defining Callbacks

```
lrd = ReduceLROnPlateau(monitor = 'val_loss',patience = 2,verbose = 1,factor = 0.8,  
min_lr = 1e-6)
```

```
mcp = ModelCheckpoint('DenseNet169.h5')
```

```
es = EarlyStopping(verbose=1, patience=2)
```

//Compiling the model

```
model.compile(optimizer='Adam', loss="categorical_crossentropy",  
metrics=METRICS)
```

//Calculating the step sizes for train and valid generators

```
STEP_SIZE_TRAIN = train_generator.n//train_generator.batch_size
```

```
STEP_SIZE_VALID = valid_generator.n//valid_generator.batch_size
```

//Running the model

```
%time
```

```
history =
```

```
model.fit_generator(generator=train_generator,steps_per_epoch=STEP_SIZE_TRAIN  
,validation_data=valid_generator,validation_steps=STEP_SIZE_VALID,epochs=10,c  
allbacks=[lrd,mcp,es])
```

//CODE FOR PREDICTION OF DR ON REDUCED DATASET WITH 2433 SAMPLES OF 5 LEVELS

//file1.csv consists of 2433 samples belonging to 5 levels

// 0- 599 1- 370 2- 976 3- 193 4- 295

//Reading csv file using pandas

```
train = pd.read_csv(r'C:\Users\DELL\Desktop\APTOS_DR\Trying\file1.csv')
```

//Printing number of samples and first 5 samples in CSV file

```
print('Number of train samples: ', train.shape[0])
```

```
display(train.head())
```

//Returns number of unique classes in diagnosis column in csv file

```
N_CLASSES = train['diagnosis'].nunique()
N_CLASSES
```

// Formatting image name and converting numeric type of diagnosis to string

```
train["id_code"] = train["id_code"].apply(lambda x: x + ".png")
train['diagnosis'] = train['diagnosis'].astype('str')
train.head()
```

//Creating Image Data Generator object

```
train_datagen=ImageDataGenerator(rescale=1./255,
                                  validation_split=0.2,
                                  horizontal_flip=True)
```

//Creating train and test generator based on validation split specified

//Train Generator

```
train_generator=train_datagen.flow_from_dataframe(
    dataframe=train,
    directory=r"C:\Users\DELL\Desktop\APTOS_DR\aptos2019-blindness-
detection/train_images/",
    x_col="id_code",
    y_col="diagnosis",
    batch_size=16,
    class_mode="categorical",
    target_size=(224, 224),
    subset='training')
```

//Validation generator

```
valid_generator=train_datagen.flow_from_dataframe(
    dataframe=train,
    directory=r"C:\Users\DELL\Desktop\APTOS_DR\aptos2019-blindness-
detection/train_images/",
    x_col="id_code",
    y_col="diagnosis",
    batch_size=16,
```

```
class_mode="categorical",
target_size=(224, 224),
subset='validation')
```

//DenseNet169 Model

```
from keras.applications.densenet import DenseNet169

base_model = DenseNet169(input_shape=(224, 224, 3), include_top=False,
weights='imagenet')
```

```
Epoch 1/10
121/121 [=====] - 2098s 17s/step - loss: 1.3579 - accuracy: 0.4675 - precision: 0.5485 - recall: 0.223
6 - auc: 0.7576 - val_loss: 0.6543 - val_accuracy: 0.8000 - val_precision: 0.8780 - val_recall: 0.7646 - val_auc: 0.9444
Epoch 2/10
121/121 [=====] - 1582s 13s/step - loss: 1.1259 - accuracy: 0.5502 - precision: 0.6511 - recall: 0.369
7 - auc: 0.8367 - val_loss: 0.7376 - val_accuracy: 0.7667 - val_precision: 0.9153 - val_recall: 0.6979 - val_auc: 0.9302
Epoch 3/10
121/121 [=====] - 1497s 12s/step - loss: 1.0301 - accuracy: 0.5918 - precision: 0.6828 - recall: 0.424
3 - auc: 0.8654 - val_loss: 0.5831 - val_accuracy: 0.8167 - val_precision: 0.9447 - val_recall: 0.7479 - val_auc: 0.9628
Epoch 4/10
121/121 [=====] - 1498s 12s/step - loss: 0.9567 - accuracy: 0.6318 - precision: 0.7178 - recall: 0.474
8 - auc: 0.8843 - val_loss: 0.4663 - val_accuracy: 0.8646 - val_precision: 0.9393 - val_recall: 0.8062 - val_auc: 0.9725
Epoch 5/10
121/121 [=====] - 1483s 12s/step - loss: 0.9025 - accuracy: 0.6505 - precision: 0.7185 - recall: 0.513
8 - auc: 0.8968 - val_loss: 0.4816 - val_accuracy: 0.8833 - val_precision: 0.9470 - val_recall: 0.8188 - val_auc: 0.9738
Epoch 6/10
121/121 [=====] - 1501s 12s/step - loss: 0.8920 - accuracy: 0.6412 - precision: 0.7079 - recall: 0.509
1 - auc: 0.8987 - val_loss: 0.4268 - val_accuracy: 0.8792 - val_precision: 0.9201 - val_recall: 0.8396 - val_auc: 0.9742
Epoch 7/10
121/121 [=====] - 1036s 9s/step - loss: 0.8096 - accuracy: 0.6797 - precision: 0.7480 - recall: 0.5757
- auc: 0.9165 - val_loss: 0.4347 - val_accuracy: 0.8833 - val_precision: 0.9131 - val_recall: 0.8542 - val_auc: 0.9736
Epoch 8/10
121/121 [=====] - ETA: 0s - loss: 0.7905 - accuracy: 0.6854 - precision: 0.7427 - recall: 0.5840 - au
c: 0.9201
Epoch 00008: ReduceLROnPlateau reducing learning rate to 0.0008000000037997961.
121/121 [=====] - 887s 7s/step - loss: 0.7905 - accuracy: 0.6854 - precision: 0.7427 - recall: 0.5840
- auc: 0.9201 - val_loss: 0.4327 - val_accuracy: 0.8625 - val_precision: 0.9040 - val_recall: 0.8438 - val_auc: 0.9759
Epoch 00008: early stopping
```

//DenseNet121 Model

```
from keras.applications.densenet import DenseNet121

base_model = DenseNet121(input_shape=(224, 224, 3), include_top=False,
weights='imagenet')
```

```
Epoch 1/10
121/121 [=====] - 1922s 16s/step - loss: 1.4039 - accuracy: 0.4384 - precision: 0.5557 - recall: 0.186
7 - auc: 0.7298 - val_loss: 0.7909 - val_accuracy: 0.7708 - val_precision: 0.8792 - val_recall: 0.7583 - val_auc: 0.9039
Epoch 2/10
121/121 [=====] - 1674s 14s/step - loss: 1.1784 - accuracy: 0.5439 - precision: 0.6324 - recall: 0.326
6 - auc: 0.8203 - val_loss: 0.6835 - val_accuracy: 0.7750 - val_precision: 0.9580 - val_recall: 0.7604 - val_auc: 0.9332
Epoch 3/10
121/121 [=====] - 1523s 13s/step - loss: 1.0903 - accuracy: 0.5788 - precision: 0.6481 - recall: 0.387
9 - auc: 0.8477 - val_loss: 0.8037 - val_accuracy: 0.7708 - val_precision: 0.9115 - val_recall: 0.7083 - val_auc: 0.9160
Epoch 4/10
121/121 [=====] - 1522s 13s/step - loss: 1.0083 - accuracy: 0.6136 - precision: 0.6803 - recall: 0.457
1 - auc: 0.8707 - val_loss: 0.6596 - val_accuracy: 0.8021 - val_precision: 0.9119 - val_recall: 0.7333 - val_auc: 0.9431
Epoch 5/10
121/121 [=====] - 1467s 12s/step - loss: 0.9541 - accuracy: 0.6183 - precision: 0.6960 - recall: 0.485
7 - auc: 0.8841 - val_loss: 0.5908 - val_accuracy: 0.8229 - val_precision: 0.9075 - val_recall: 0.7563 - val_auc: 0.9566
Epoch 6/10
121/121 [=====] - 1524s 13s/step - loss: 0.9091 - accuracy: 0.6453 - precision: 0.7170 - recall: 0.520
5 - auc: 0.8950 - val_loss: 0.4623 - val_accuracy: 0.8708 - val_precision: 0.9056 - val_recall: 0.8396 - val_auc: 0.9719
Epoch 7/10
121/121 [=====] - 1113s 9s/step - loss: 0.8810 - accuracy: 0.6526 - precision: 0.7164 - recall: 0.5398
- auc: 0.9011 - val_loss: 0.8019 - val_accuracy: 0.7875 - val_precision: 0.8203 - val_recall: 0.7229 - val_auc: 0.9227
Epoch 8/10
121/121 [=====] - ETA: 0s - loss: 0.8311 - accuracy: 0.6708 - precision: 0.7356 - recall: 0.5715 - au
c: 0.9120
Epoch 00008: ReduceLROnPlateau reducing learning rate to 0.0008000000037997961.
121/121 [=====] - 922s 8s/step - loss: 0.8311 - accuracy: 0.6708 - precision: 0.7356 - recall: 0.5715
- auc: 0.9120 - val_loss: 0.4752 - val_accuracy: 0.8604 - val_precision: 0.9048 - val_recall: 0.8313 - val_auc: 0.9691
Epoch 00008: early stopping
```


//ResNet152 Model

```
base_model =  
tf.keras.applications.ResNet152V2(input_shape=(224,224,3),include_top=False,weights="imagenet")
```

```
Epoch 1/10  
121/121 [=====] - 2155s 18s/step - loss: 1.1518 - accuracy: 0.5595 - precision: 0.6419 - recall: 0.363  
5 - auc: 0.8274 - val_loss: 0.8264 - val_accuracy: 0.7437 - val_precision: 0.8571 - val_recall: 0.6500 - val_auc: 0.9114  
Epoch 2/10  
121/121 [=====] - 1611s 13s/step - loss: 1.1451 - accuracy: 0.5450 - precision: 0.6117 - recall: 0.363  
0 - auc: 0.8288 - val_loss: 0.4033 - val_accuracy: 0.8604 - val_precision: 0.9309 - val_recall: 0.8417 - val_auc: 0.9788  
Epoch 3/10  
121/121 [=====] - 1518s 13s/step - loss: 1.1121 - accuracy: 0.5554 - precision: 0.6308 - recall: 0.390  
0 - auc: 0.8391 - val_loss: 0.6761 - val_accuracy: 0.7729 - val_precision: 0.8667 - val_recall: 0.7312 - val_auc: 0.9422  
Epoch 4/10  
121/121 [=====] - 1547s 13s/step - loss: 1.0949 - accuracy: 0.5653 - precision: 0.6367 - recall: 0.408  
2 - auc: 0.8438 - val_loss: 0.1157 - val_accuracy: 0.9667 - val_precision: 0.9829 - val_recall: 0.9563 - val_auc: 0.9974  
Epoch 5/10  
121/121 [=====] - 1512s 12s/step - loss: 1.0670 - accuracy: 0.5798 - precision: 0.6557 - recall: 0.425  
9 - auc: 0.8518 - val_loss: 0.2761 - val_accuracy: 0.9250 - val_precision: 0.9416 - val_recall: 0.9062 - val_auc: 0.9887  
Epoch 6/10  
121/121 [=====] - ETA: 0s - loss: 1.0231 - accuracy: 0.5996 - precision: 0.6726 - recall: 0.4550 - au  
c: 0.8640  
Epoch 00006: ReduceLROnPlateau reducing learning rate to 0.0008000000037997961.  
121/121 [=====] - 1530s 13s/step - loss: 1.0231 - accuracy: 0.5996 - precision: 0.6726 - recall: 0.455  
0 - auc: 0.8640 - val_loss: 0.4687 - val_accuracy: 0.8417 - val_precision: 0.8891 - val_recall: 0.8021 - val_auc: 0.9711  
Epoch 00006: early stopping
```

//Tuning pre-trained models

```
for layer in base_model.layers[:10]:
```

```
    layer.trainable=False
```

//Adding tuned pre-trained model to Sequential model

```
model=Sequential()  
model.add(base_model)  
model.add(Dropout(0.5))  
model.add(Flatten())  
model.add(BatchNormalization())  
model.add(Dense(256,kernel_initializer='he_uniform'))  
model.add(BatchNormalization())  
model.add(Activation('relu'))  
model.add(Dropout(0.5))  
model.add(Dense(128,kernel_initializer='he_uniform'))  
model.add(BatchNormalization())  
model.add(Activation('relu'))  
model.add(Dropout(0.5))  
model.add(Dense(32,kernel_initializer='he_uniform'))  
model.add(BatchNormalization())
```

```
model.add(Activation('relu'))
model.add(Dense(5,activation='softmax'))
```

//Defining Metrics

```
METRICS = [
    tf.keras.metrics.CategoricalAccuracy(name='accuracy'),
    tf.keras.metrics.Precision(name='precision'),
    tf.keras.metrics.Recall(name='recall'),
    tf.keras.metrics.AUC(name='auc'),
]
```

//Defining Callbacks

```
lrd = ReduceLROnPlateau(monitor = 'val_loss',patience = 2,verbose = 1,factor = 0.8,
min_lr = 1e-6)
mcp = ModelCheckpoint('DenseNet169.h5')
es = EarlyStopping(verbose=1, patience=2)
```

//Compiling the model

```
model.compile(optimizer='Adam', loss="categorical_crossentropy",
metrics=METRICS)
```

//Calculating the step sizes for train and valid generators

```
STEP_SIZE_TRAIN = train_generator.n//train_generator.batch_size
STEP_SIZE_VALID = valid_generator.n//valid_generator.batch_size
```

//Running the model

```
%time
history =
model.fit_generator(generator=train_generator,steps_per_epoch=STEP_SIZE_TRAIN
,validation_data=valid_generator,validation_steps=STEP_SIZE_VALID,epochs=10,c
allbacks=[lrd,mcp,es])
```

//CODE FOR EMSEMBLING THE MODELS

//Loading the saved models

```
from keras.models import load_model
from sklearn.metrics import accuracy_score
model1 = load_model('DenseNet121.hdf5')
model2=load_model('DenseNet169.hdf5')
model3=load_model('ResNet152.hdf5')
```

//Creating data generators and formatting image names

```
from keras.preprocessing.image import ImageDataGenerator
import pandas as pd
```

//Formatting

```
train = pd.read_csv(r'C:\Users\DELL\Desktop\APTOS_DR\Trying\file1.csv')
train["id_code"] = train["id_code"].apply(lambda x: x + ".png")
train['diagnosis'] = train['diagnosis'].astype('str')
```

//Train and valid data generators

```
train_datagen=ImageDataGenerator(rescale=1./255,
                                  validation_split=0.2,
                                  horizontal_flip=True)
```

```
train_generator=train_datagen.flow_from_dataframe(
    dataframe=train,
    directory=r"C:\Users\DELL\Desktop\APTOS_DR\aptos2019-blindness-
detection/train_images/",
    x_col="id_code",
    y_col="diagnosis",
    batch_size=16,
    class_mode="categorical",
    target_size=(224, 224),
    subset='training')
```

```

valid_generator=train_datagen.flow_from_dataframe(
    dataframe=train,
    directory=r"C:\Users\DELL\Desktop\APTOS_DR\aptos2019-blindness-
detection/train_images/",
    x_col="id_code",
    y_col="diagnosis",
    batch_size=16,
    class_mode="categorical",
    target_size=(224, 224),
    subset='validation')

```

//Splitting train and valid generators into x and y

```

x_train, y_train = next(train_generator)
x_test, y_test = next(valid_generator)

```

//Converting into numpy arrays

```

import numpy as np
y_train=np.argmax(y_train,axis=1)
y_test=np.argmax(y_test,axis=1)

```

//List of Models

```

models=[model1,model2,model3]

```

// Model Averaging Ensemble

```

preds = [model.predict(x_test) for model in models]
preds=np.array(preds)
summed = np.sum(preds, axis=0)
ensemble_prediction = np.argmax(summed, axis=1)
prediction1 = model1.predict_classes(x_test)
prediction2 = model2.predict_classes(x_test)
prediction3 = model3.predict_classes(x_test)
accuracy1 = accuracy_score(y_test, prediction1)

```

```

accuracy2 = accuracy_score(y_test, prediction2)
accuracy3 = accuracy_score(y_test, prediction3)
ensemble_accuracy = accuracy_score(y_test, ensemble_prediction)
print('Accuracy Score for model1 = ', accuracy1)
print('Accuracy Score for model2 = ', accuracy2)
print('Accuracy Score for model3 = ', accuracy3)
print('Accuracy Score for average ensemble = ', ensemble_accuracy)

```

```

Accuracy Score for model1 = 0.875
Accuracy Score for model2 = 0.875
Accuracy Score for model3 = 0.875
Accuracy Score for average ensemble = 0.875

```

//Weighted Average Ensemble

```

models = [model1, model2, model3]
preds = [model.predict(x_test) for model in models]
preds=np.array(preds)
weights = [0.0, 0.0, 1.0]
#Use tensordot to sum the products of all elements over specified axes.
weighted_preds = np.tensordot(preds, weights, axes=((0),(0)))
weighted_ensemble_prediction = np.argmax(weighted_preds, axis=1)
weighted_accuracy = accuracy_score(y_test, weighted_ensemble_prediction)
print('Accuracy Score for model1 = ', accuracy1)
print('Accuracy Score for model2 = ', accuracy2)
print('Accuracy Score for model3 = ', accuracy3)
print('Accuracy Score for average ensemble = ', ensemble_accuracy)
print('Accuracy Score for weighted average ensemble = ', weighted_accuracy)

```

```

Accuracy Score for model1 = 0.875
Accuracy Score for model2 = 0.875
Accuracy Score for model3 = 0.875
Accuracy Score for average ensemble = 0.875
Accuracy Score for weighted average ensemble = 0.875

```

CHAPTER-8

RESULTED OUTPUTS

//Observed Output when all 3662 samples of 5 levels are considered

// 0- 1805 1- 370 2- 999 3- 193 4- 295

Model	Accuracy	Precision	Recall	AUC
DenseNet169	80.69	85.28	77.22	0.9620
DenseNet121	79.31	84.18	73.89	0.9584
ResNet152	75.42	80.88	69.31	0.9423

//Observed Output when all 3662 samples of only 2 levels are considered

// 0- 1805 1- 1857

Model	Accuracy	Precision	Recall	AUC
DenseNet169	97.08	98.68	95.90	0.9956
DenseNet121	96.53	98.41	95.13	0.9929
ResNet152	92.92	97.46	89.18	0.9724

//Observed Output when only 2433 samples among 3662 samples are considered by reducing the dataset

// 0- 599 1- 370 2- 976 3- 193 4- 295

Model	Accuracy	Precision	Recall	AUC
DenseNet169	86.25	90.40	84.38	0.9759
DenseNet121	86.04	90.48	83.13	0.9691
ResNet152	84.17	88.91	80.21	0.9711

//Results obtained by Ensemble of models applied on the 3 models trained on 2433 samples of 5 levels

Model Averaging Ensemble	
Model	Accuracy
DenseNet169	87.5
DenseNet121	87.5
ResNet152	87.5
Average Ensemble accuracy = 87.5	

Weighted Average Ensemble	
Model	Accuracy
DenseNet169	87.5
DenseNet121	87.5
ResNet152	87.5
Weighted Average Ensemble accuracy = 87.5	

CHAPTER-9

CONCLUSION AND FUTURE SCOPE

9.1 CONCLUSION

Five stages of Diabetic Retinopathy are chosen for testing and validation of image classification using deep learning. The convolutional neural network is used in all the 3 architectures (ResNet152, DenseNet121,DenseNet169) for classification purpose. From the experiments, it is observed that the images are classified correctly and shows the effectiveness of deep learning algorithm.

We considered the complete Aptos dataset which contains 3662 images belonging to 5 classes. Then we processed our dataset on ResNet152, we got an accuracy of 75% .Next we processed the same dataset on DenseNet121 and got an accuracy of 79%.Later, we again processed the same dataset on DenseNet169 and got an accuracy of 80%.

We considered the complete Aptos dataset which contains 3663 images and merged the stages which resulted in 2 classes. Then we processed our dataset on ResNet152, we got an accuracy of 92%. Next we processed the same dataset on DenseNet121 and got an accuracy of 96%. Later, we again processed the same dataset on DenseNet169 and got an accuracy of 97%.

A new dataset is formed by considering 2433 fundus images from the Aptos dataset which contains 3662 images belonging to 5 classes. Then we processed our dataset on ResNet152, we got an accuracy of 84% .Next we processed the same dataset on DenseNet121 and got an accuracy of 86%. Later, we again processed the same dataset on DenseNet169 and got an accuracy of 86%.

As a final aim of ensemble of models, we used the models that are created on 2433 images belonging to 5 classes. In this, we use two types of ensemble techniques. The first one is model averaging ensemble through which we got an resultant accuracy of 87%. The second technique is weighted averaging ensemble with which we got an accuracy of 87% which is same as result of model averaging ensemble.

9.2 FUTURE SCOPE

In future, we will run our model on Kaggle dataset which contains more than thousands of images in order to improve the accuracy. The outcomes in this research are based on results that involve only sample datasets. It is necessary that additional datasets should be considered for the evaluation of different classification problems as the information growth in the recent technology is extending to heights beyond assumptions. Recent field of technology is growing and data are by nature dynamic.

The scope of future work can deal with Incremental learning, which stores the existing model and processes the new incoming data more efficiently.

The image classification model can be enhanced in future, by including more low-level features such as shape and spatial location features apart from optimizing the weights and learning rate of the neural network. In this research work, while evaluating the fitness of an individual, the consideration is given only to the occurrence frequencies of an image in retrieval result and not the location of it.

REFERENCES

- [1] S. Qummar et al.: Deep Learning Ensemble Approach for DR Detection Sehrish Qummar, Fiaz Gul Khan, Sajid Shah , Ahmad Khan , Shahaboddin Shamshirband , Zia Ur Rehman , Iftikhar Ahmed Khan , And Waqas Jadoon Volume 7, 2019
- [2] Diabetic Retinopathy Detection via Deep Convolutional Networks for Discriminative Localization and Visual Explanation Zhiguang Wang Jianbo Yang The Workshops of the Thirty-Second AAAI Conference on Artificial Intelligence
- [3] Diabetic Retinopathy Detection using Machine Learning Revathy R , Nithya B , Reshma , Ragendhu, Sumithra M D Vol. 9 Issue 06, June-2020
- [4] A convolutional neural network for the screening and staging of diabetic retinopathy Mohamed Shaban1 , Zeliha Ogur , Ali Mahmoud , Andrew Switala , Ahmed Shalaby2, Hadil Abu Khalifeh, Mohammed Ghazal, Luay FraiwanID , Guruprasad Giridharan, Harpal Sandhu , Ayman S. El-BazID June 22, 2020
- [5] M. D. Abramoff, P. T. Lavin, M. Birch, N. Shah, and J. C. Folk, “Pivotal trial of an autonomous AI-based diagnostic system for detection of diabetic retinopathy in primary care offices,” *Npj Digit. Med.*, vol. 1, no. 1, p. 39, Aug. 2018.
- [6] L. Seoud, T. Hurtut, J. Chelbi, F. Cheriet, and J. M. P. Langlois, “Red lesion detection using dynamic shape features for diabetic retinopathy screening,” *IEEE Trans. Med. Imag.*, vol. 35, no. 4, pp. 1116–1126, Apr. 2016.
- [7] R. A. Welikala, M. M. Fraz, J. Dehmeshki, A. Hoppe, V. Tah, S. Mann, and T. H. Williamson, “Genetic algorithm based feature selection combined with dual classification for the automated detection of proliferative diabetic retinopathy,” *Computerized Med. Imag. Graph.*, vol. 43pp. 64–77, Jul. 2015.
- [8] R. Gargeya and T. Leng, “Automated identification of diabetic retinopathy using deep learning,” *Ophthalmology*, vol. 124, no. 7, pp. 962–969, 2017.
- [9] Antony, M., and Brggemann, S. 2015. Kaggle Diabetic Retinopathy Detection: Team o O solution.
- [10] Bazzani, L.; Bergamo, A.; Anguelov, D.; and Torresani, L. 2016. Selftaught object alization with deep networks. In 2016 IEEE Winter Conference on Applications of Computer Vision (WACV), 1–9.
- [11] Deng, L. 2014. A tutorial survey of architectures, algorithms, and applications for deep learning. *APSIPA Transactions on Signal and Information Processing*.
- [12] Dosovitskiy, A., and T. Brox. 2016. Inverting visual representations with convolutional networks. In *CVPR.2015. Diabetic Retinopathy Detection*

Kaggle.Krizhevsky, A.; Sutskever, I.; and Hinton, G. E. 2012.Imagenet classification with deep convolutional neural networks. In NIPS.

[13] LeCun, Y.; Boser, B.; Denker, J. S.; Henderson, D.; Howard,R. E.; Hubbard, W.; and Jackel, L. D. 1989. Backpropagation applied to handwritten zip code recognition. *Neural Computation* 1(4):541–551.

[14] LeCun, Y.; Bottou, L.; Orr, G.; and Muller, K. 1998. Efficient backprop. *Neural Networks: Tricks of the trade* 950.

[15] Lim, G.; Lee, M. L.; Hsu, W.; and Wong, T. Y. 2014. Transformed representations for convolutional neural networks in diabetic retinopathy screening. In *AAAI Workshop on Modern Artificial Intelligence for Health Analytics (MAIHA)*,AAAI.

[16] Mahendran, A., and Vedaldi, A. 2015. Understanding deep image representations by inverting them. In *CVPR*.

[17] Farrikh Alzami, Abdussalam, Rama Arya Megantara and Ahmad Zainul Fanani, “Diabetic Retinopathy Grade Classification based on Fractal Analysis and Random Forest”, *International Seminar on Application for Technology of Information and Communication*, 2019.

[18] Dinial Utami Nurul Qomariah, Handayani Tjandrasa and Chastine Fatichah, “Classification of Diabetic Retinopathy and Normal Retinal Images using CNN and SVM”, *12th International Conference on Information and Communication Technology and System*, 2019.

[19] Shailesh Kumar and Basant Kumar “Diabetic Retinopathy Detection by Extracting Area and Number of Microaneurysms from Colour Fundus Images”, *5th International Conference on Signal Processing and Integrated Networks*, 2018.

[20] Mohamed Chetoui, Moulay A Akhloufi, Mustapha Kardoucha , “Diabetic Retinopathy Detection using Machine Learning and Texture Features”, *IEEE Canadian Conference on Electrical and Computer Engineering*, 2018.

[21] S Choudhury, S Bandyopadhyay, SK Latib, DK Kole, C Giri, “Fuzzy C Means based Feature Extraction and Classifiaction of Diabetic Retinopathy using Support Vector Machines”, *International Conference on Communication and Signal Processing*, April 2016.

[22] Surbhi Sangwan, Vishal Sharma, Misha Kakkar, “Identification of Different Stages of Diabetic Retinopathy” *International Conference on Computer and Computational Sciences*, 2015.

[23] Morium Akter, Mohammed Shorif Uddin, Mahmudul Hasan Khan, “Morphology based Exudate Detection from Colour Fundus Images in Diabetic Retinopathy” *International Conference on Electrical*.

- [24] Litjens G., Kooi T., Benjnordi B., Setio A., Ciompi F., Ghafoorian M., et al., A Survey on Deep Learning on Medical Image Analysis, *Medical Image Analysis*, vol. 42, pp. 60–88, 2017.
- [25] O’Shea K. and Nash R., *An Introduction to Convolutional Neural Networks, Neural and Evolutionary Computing*, Cornell University Library, 2016.
- [26] Mookiah M., Acharya U., Chua C., Lim C., Ng E. and Laude A., Computer-Aided Diagnosis of Diabetic Retinopathy: A Review, *Computers in Biology and Medicine*, vol. 43, no. 12, pp. 2136–2155, 2013.
- [27] Acharya U., Chua C., Ng E., Yu W., Chee C., Application of Higher Order Spectra for the Identification of Diabetes Retinopathy Stages, *Journal of Medical Systems*, vol. 32, no. 6, pp. 481–488, 2008.
- [28] Acharya U., Lim C., Ng E., Chee C. and Tamura T., Computer-Based Detection of Diabetes Retinopathy Stages using Digital Fundus Images, *Proceedings of the Institution of Mechanical Engineers*, vol.223, no. 5, pp. 545–553, 2009.
- [29] Nayak J., Bhat P., Acharya R., Lim C. and Kagathi M., Automated Identification of Diabetic Retinopathy Stages using Digital Fundus Images, *Journal of Medical Systems*, vol. 32, no. 2, pp. 107–115, 2008.
- [30] H. Pratt, F. Coenen, D. Broadbent, S. Harding, Y. Zheng, “Convolutional Neural Networks for Diabetic Retinopathy”, *International Conference on Medical Imaging Understanding and Analysis*, Loughborough, UK, July 2016.